

transformer

transformer를 이야기 하기 전에 먼저 attention에 대해서 알아야 한다...

RNN에 기반한 seq2seq 모델에는 크게 두 가지 문제가 있다.

첫째, 하나의 고정된 크기의 벡터에 모든 정보를 압축하려고 하니까 정보 손실이 발생합니다.

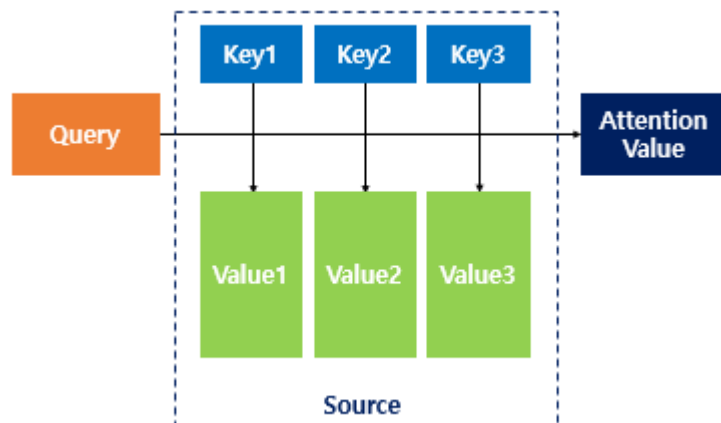
둘째, RNN의 고질적인 문제인 기울기 소실(vanishing gradient) 문제가 존재합니다.

결국 이는 기계 번역 분야에서 입력 문장이 길면 번역 품질이 떨어지는 현상으로 나타났습니다. 이를 위한 대안으로 입력 시퀀스가 길어지면 출력 시퀀스의 정확도가 떨어지는 것을 보정해주기 위한 등장한 기법이 어텐션(attention)이다.

1. 어텐션(Attention)의 아이디어

어텐션의 기본 아이디어는 디코더에서 출력 단어를 예측하는 매 시점(time step)마다, 인코더에서의 전체 입력 문장을 다시 한 번 참고한다는 점입니다. 단, 전체 입력 문장을 전부 다 동일한 비율로 참고하는 것이 아니라, 해당 시점에서 예측해야 할 단어와 연관이 있는 입력 단어 부분을 좀 더 집중(attention)해서 보게 됩니다.

2. 어텐션 함수(Attention Function)



어텐션을 함수로 표현하면 주로 다음과 같이 표현됩니다.

Attention(Q, K, V) = Attention Value

어텐션 함수는 주어진 '쿼리(Query)'에 대해서 모든 '키(Key)'와의 유사도를 각각 구합니다. 그리고 구해낸 이 유사도를 키와 맵핑되어있는 각각의 '값(Value)'에 반영해줍니다. 그리고 유사도가 반영된 '값(Value)'을 모두 더해서 리턴합니다. 여기서는 이를 어텐션 값(Attention Value)이라고 하겠습니다.

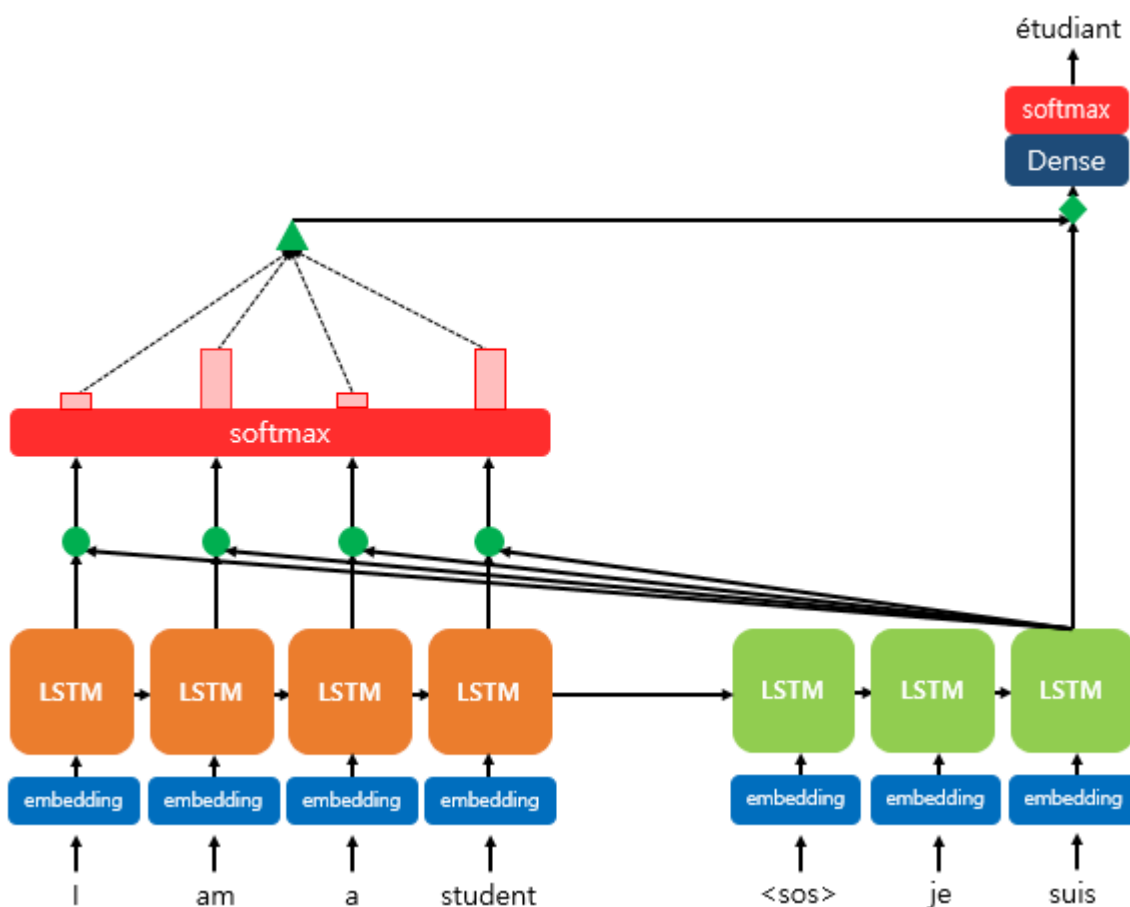
지금부터 배우게 되는 seq2seq + 어텐션 모델에서 Q, K, V에 해당되는 각각의 Query, Keys, Values는 각각 다음과 같습니다.

CopyQ = Query : t 시점의 디코더 셀에서의 은닉 상태

K = Keys : 모든 시점의 인코더 셀의 은닉 상태들

V = Values : 모든 시점의 인코더 셀의 은닉 상태들

어텐션은 다양한 종류가 있는데 그 중에서도 가장 수식적으로 이해하기 쉽게 수식을 적용한 닷-프로덕트 어텐션(Dot-Product Attention)을 다루겠다.

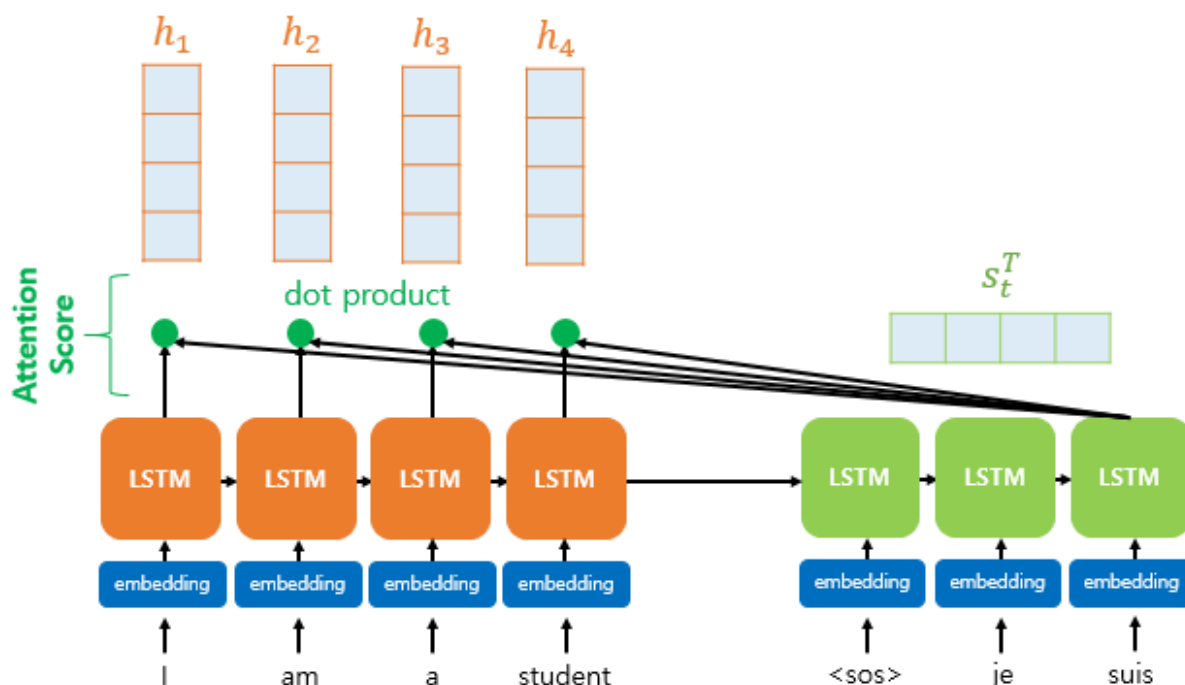


위에서는 디코더의 첫번째, 두번째 LSTM 셀은 이미 어텐션 메커니즘을 통해 je와 suis를 예측하는 과정을 거쳤다고 가정합니다. 디코더의 세번째 LSTM 셀은 출력 단어를 예측하기 위해서 인코더의 모든 입력 단어들의 정보를 다시 한번 참고하고자 합니다.

소프트맥스 함수를 통해 나온 결과값은 I, am, a, student 단어 각각이 출력 단어를 예측할 때 얼마나 도움이 되는지의 정도를 수치화한 값입니다. 위의 그림에서는 빨간 직사각형의 크기로 소프트맥스 함수의 결과값의 크기를 표현했습니다. 직사각형의 크기가 클 수록 도움이 되는 정도의 크기가 큼니다. 각 입력 단어가 디코더의 예측에 도움이 되는 정도가 수치화하여 측정되면 이를 하나의 정보로 담아서 디코더로 전송됩니다. 위의 그림에서는 초록색 삼각

형이 이에 해당됩니다. 결과적으로, 디코더는 출력 단어를 더 정확하게 예측할 확률이 높아 집니다. 좀 더 상세히 알아보겠습니다.

1) 어텐션 스코어(Attention Score)를 구한다.



인코더의 시점(time step)을 각각 1, 2, ... N이라고 하였을 때 인코더의 은닉 상태(hidden state)를 각각 $h_1, h_2, \dots, h_N$ 라고 합시다. 디코더의 현재 시점(time step) t 에서의 디코더의 은닉 상태(hidden state)를 s_t 라고 합시다. 또한 여기서는 인코더의 은닉 상태와 디코더의 은닉 상태의 차원이 같다고 가정합니다. 위의 그림의 경우에는 인코더의 은닉 상태와 디코더의 은닉 상태가 동일하게 차원이 4입니다.

어텐션 메커니즘의 첫 걸음인 어텐션 스코어(Attention score)에 대해서 배우기전에, 이전 챕터 배웠던 디코더의 현재 시점 t 에서 필요한 입력값을 다시 상기해보겠습니다. 시점 t 에서 출력 단어를 예측하기 위해서 디코더의 셀은 두 개의 입력값을 필요로 하는데, 바로 이전 시점인 $t-1$ 의 은닉 상태와 이전 시점 $t-1$ 에 나온 출력 단어입니다.

그런데 어텐션 메커니즘에서는 출력 단어 예측에 또 다른 값을 필요로 하는데 바로 어텐션 값(Attention Value)이라는 새로운 값입니다. t 번째 단어를 예측하기 위한 어텐션 값을 **at**이라고 정의하겠습니다.

어텐션 값이라는 새로운 개념이 등장한 만큼, 어텐션 값이 현재 시점 t 에서의 출력 예측에 구체적으로 어떻게 반영되는지는 뒤에서 설명하겠습니다. 지금부터 배우는 모든 과정은 **at**를 구하기 위한 여정입니다. 그리고 그 여정의 첫 걸음은 바로 어텐션 스코어(Attention Score)를 구하는 일입니다. 어텐션 스코어란 현재 디코더의 시점 t 에서 단어를 예측하기 위해, 인코더의 모든 은닉 상태 각각이 디코더의 현재 시점의 은닉 상태 s_t 와 얼마나 유사한지를 판단하는 스코어값입니다.

닷-프로덕트 어텐션에서는 이 스코어 값을 구하기 위해 s_t 를 전치(transpose)하고 각 은닉 상태와 내적(dot product)을 수행합니다. 즉, 모든 어텐션 스코어 값은 스칼라입니다. 예를 들어 s_t 와 인코더의 i 번째 은닉 상태의 어텐션 스코어의 계산 방법은 아래와 같습니다.

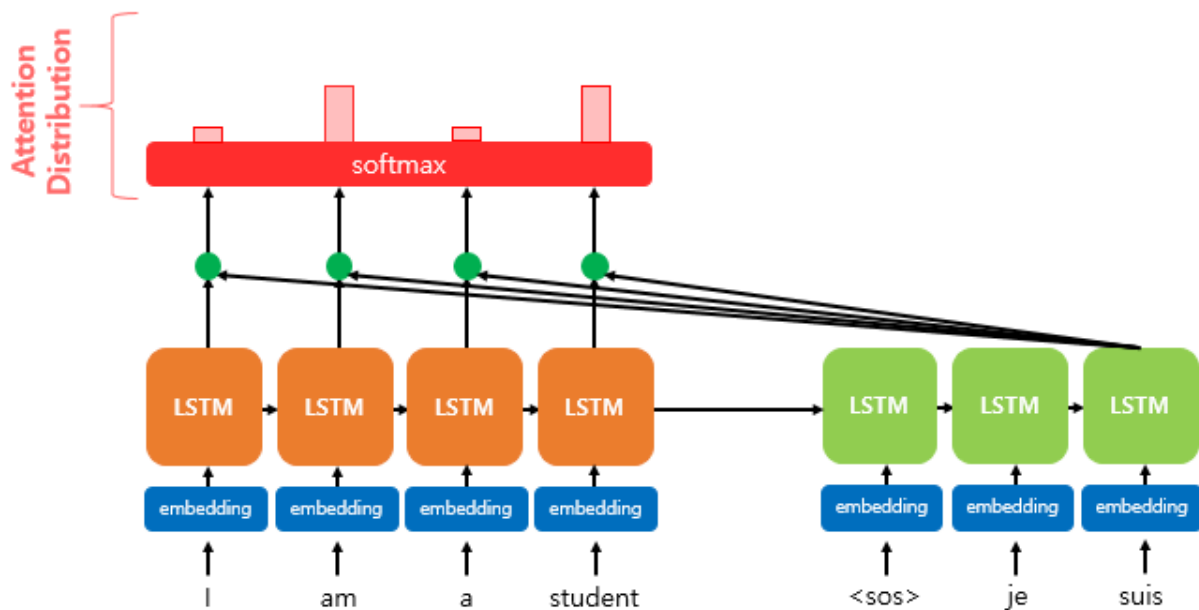
어텐션 스코어 함수를 정의해보면 다음과 같습니다.

$$score(s_t, h_i) = s_t^T h_i$$

s_t 와 인코더의 모든 은닉 상태의 어텐션 스코어의 모음값을 e^t 라고 정의하겠습니다. e^t 의 수식은 다음과 같습니다.

$$e^t = [s_t^T h_1, \dots, s_t^T h_N]$$

2) 소프트맥스(softmax) 함수를 통해 어텐션 분포(Attention Distribution)를 구한다.



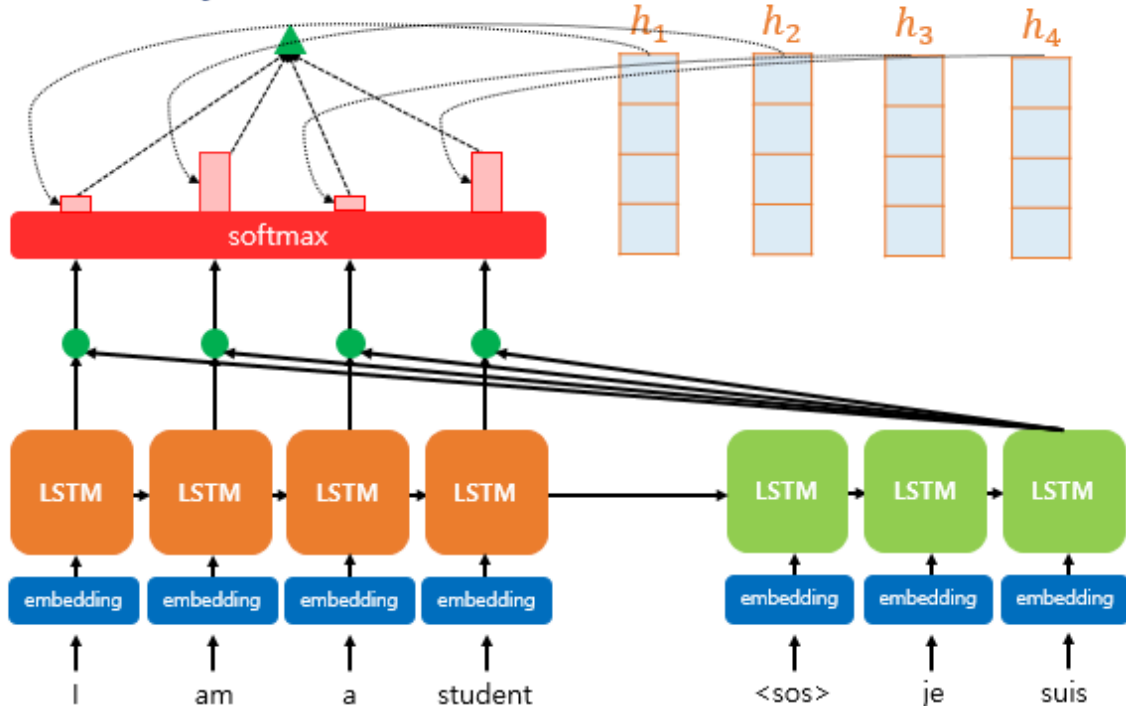
e^t 에 소프트맥스 함수를 적용하여, 모든 값을 합하면 1이 되는 확률 분포를 얻어냅니다. 이를 어텐션 분포(Attention Distribution)라고 하며, 각각의 값은 어텐션 가중치(Attention Weight)라고 합니다. 예를 들어 소프트맥스 함수를 적용하여 얻은 출력값인 'I', 'am', 'a', 'student'의 어텐션 가중치를 각각 0.1, 0.4, 0.1, 0.4라고 합시다. 이들의 합은 1입니다. 위의 그림은 각 인코더의 은닉 상태에서의 어텐션 가중치의 크기를 직사각형의 크기를 통해 시각화하였습니다. 즉, 어텐션 가중치가 클수록 직사각형이 큼니다.

디코더의 시점 t 에서의 어텐션 가중치의 모음값인 어텐션 분포를 α^t 이라고 할 때, α^t 을 식으로 정의하면 다음과 같습니다.

$$\alpha^t = \text{softmax}(e^t)$$

3) 각 인코더의 어텐션 가중치와 은닉 상태를 가중합하여 어텐션 값(Attention Value)을 구한다.

Attention Value a_t

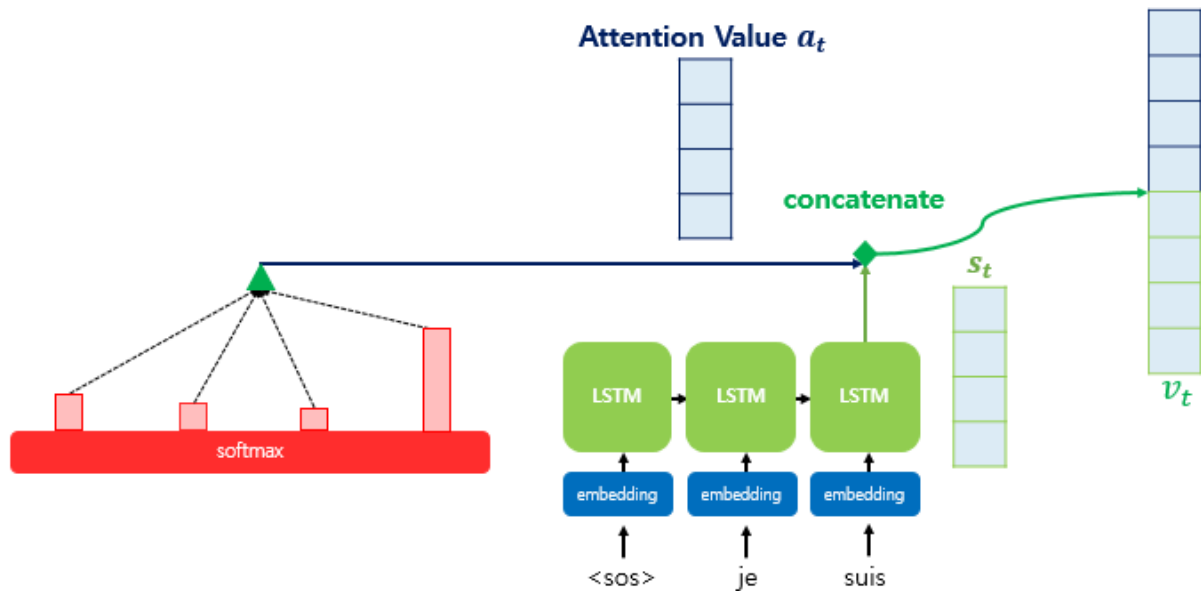


이제 지금까지 준비해온 정보들을 하나로 합치는 단계입니다. 어텐션의 최종 결과값을 얻기 위해서 각 인코더의 은닉 상태와 어텐션 가중치값들을 곱하고, 최종적으로 모두 더합니다. 요약하면 가중합(Weighted Sum)을 진행합니다. 아래는 어텐션의 최종 결과. 즉, 어텐션 함수의 출력값인 어텐션 값(Attention Value) a_t 에 대한 식을 보여줍니다.

$$a_t = \sum_{i=1}^N \alpha_i^t h_i$$

이러한 어텐션 값 a_t 은 종종 인코더의 문맥을 포함하고 있다고하여, **컨텍스트 벡터(context vector)**라고도 불립니다. 앞서 배운 가장 기본적인 seq2seq에서는 인코더의 마지막 은닉 상태를 컨텍스트 벡터라고 부르는 것과 대조됩니다.

4) 어텐션 값과 디코더의 t 시점의 은닉 상태를 연결한다.(Concatenate)



어텐션 함수의 최종값인 어텐션 값 a_t 을 구했습니다. 사실 어텐션 값이 구해지면 어텐션 메커니즘은 a_t 를 s_t 와 결합(concatenate)하여 하나의 벡터로 만드는 작업을 수행합니다. 이를 v_t 라고 정의해보겠습니다. 그리고 이 v_t 를 y^* 예측 연산의 입력으로 사용하셔서 인코더로부터 얻은 정보를 활용하여 y^* 를 좀 더 잘 예측할 수 있게 됩니다. 이것이 어텐션 메커니즘의 핵심입니다.

5) 출력층 연산의 입력이 되는 $\tilde{s}_t$ 를 계산합니다.

$$\tanh \left(W_c \times v_t \right) = \tilde{s}_t$$

논문에서는 v_t 를 바로 출력층으로 보내기 전에 신경망 연산을 한 번 더 추가하였습니다. 가중치 행렬과 곱한 후에 하이퍼볼릭탄젠트 함수를 지나도록 하여 출력층 연산을 위한 새로운 벡터인 $\tilde{s}_t$ 를 얻습니다. 어텐션 메커니즘을 사용하지 않는 seq2seq에서는 출력층의 입력이 t 시점의 은닉 상태인 s_t 였던 반면, 어텐션 메커니즘에서는 출력층의 입력이 $\tilde{s}_t$ 가 되는 셈입니다.

식으로 표현하면 다음과 같습니다. W_c 는 학습 가능한 가중치 행렬, b_c 는 편향입니다. 그림에서 편향은 생략했습니다.

$$\tilde{s}_t = \tanh(\mathbf{W}_c[a_t; s_t] + b_c)$$

6) $s_{\sim t}$ 를 출력층의 입력으로 사용합니다.

$s_{\sim t}$ 를 출력층의 입력으로 사용하여 예측 벡터를 얻습니다.

$$\hat{y}_t = \text{Softmax}(W_y \tilde{s}_t + b_y)$$

1. 기존의 seq2seq 모델의 한계

트랜스포머에 대해서 배우기 전에 기존의 seq2seq를 상기해봅시다. 기존의 seq2seq 모델은 인코더-디코더 구조로 구성되어 있었습니다. 여기서 인코더는 입력 시퀀스를 하나의 벡터 표현으로 압축하고, 디코더는 이 벡터 표현을 통해서 출력 시퀀스를 만들어냈습니다. 하지만 이러한 구조는 인코더가 입력 시퀀스를 하나의 벡터로 압축하는 과정에서 입력 시퀀스의 정보가 일부 손실된다는 단점이 있었고, 이를 보정하기 위해 어텐션이 사용되었습니다. 그런데 어텐션을 RNN의 보정을 위한 용도로서 사용하는 것이 아니라 어텐션만으로 인코더와 디코더를 만들어보면 어떨까요?

2. 트랜스포머(Transformer)의 주요 하이퍼파라미터

시작에 앞서 트랜스포머의 하이퍼파라미터를 정의합니다. 각 하이퍼파라미터의 의미에 대해서는 뒤에서 설명하기로하고, 여기서는 트랜스포머에는 이러한 하이퍼파라미터가 존재한다는 정도로만 이해해보겠습니다. 아래에서 정의하는 수치는 트랜스포머를 제안한 논문에서 사용한 수치로 하이퍼파라미터는 사용자가 모델 설계시 임의로 변경할 수 있는 값들입니다.

`dmodel=512`

트랜스포머의 인코더와 디코더에서의 정해진 입력과 출력의 크기를 의미합니다. 임베딩 벡터의 차원 또한 `dmodel`이며, 각 인코더와 디코더가 다음 층의 인코더와 디코더로 값을 보낼 때에도 이 차원을 유지합니다. 논문에서는 512입니다.

`num_layers=6`

트랜스포머에서 하나의 인코더와 디코더를 층으로 생각하였을 때, 트랜스포머 모델에서 인코더와 디코더가 총 몇 층으로 구성되었는지를 의미합니다. 논문에서는 인코더와 디코더를 각각 총 6개 쌓았습니다.

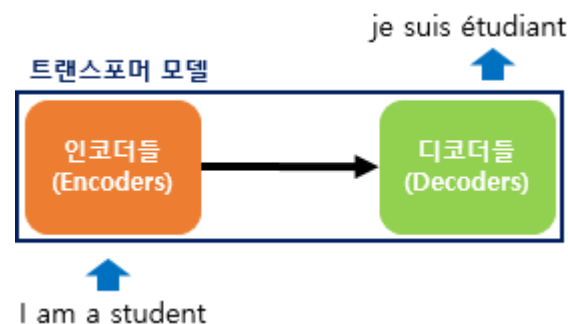
`num_heads=8`

트랜스포머에서는 어텐션을 사용할 때, 한 번 하는 것 보다 여러 개로 분할해서 병렬로 어텐션을 수행하고 결과값을 다시 하나로 합치는 방식을 택했습니다. 이때 이 병렬의 개수를 의미합니다.

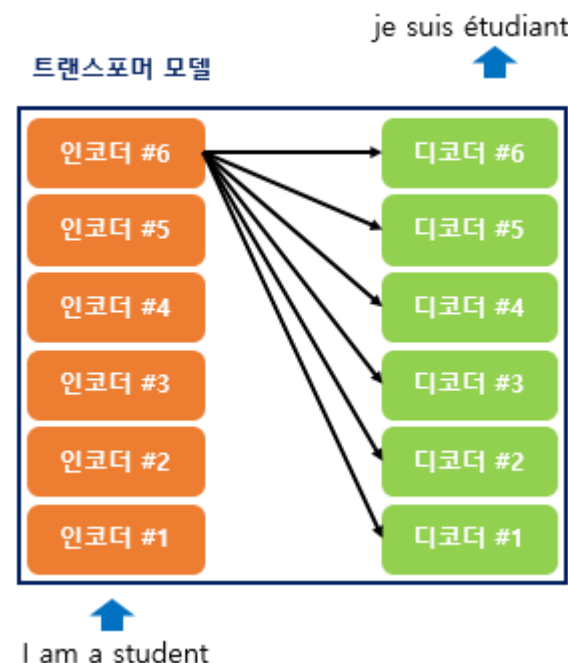
$d_{ff}=2048$

트랜스포머 내부에는 피드 포워드 신경망이 존재하며 해당 신경망의 은닉층의 크기를 의미합니다. 피드 포워드 신경망의 입력층과 출력층의 크기는 d_{model} 입니다.

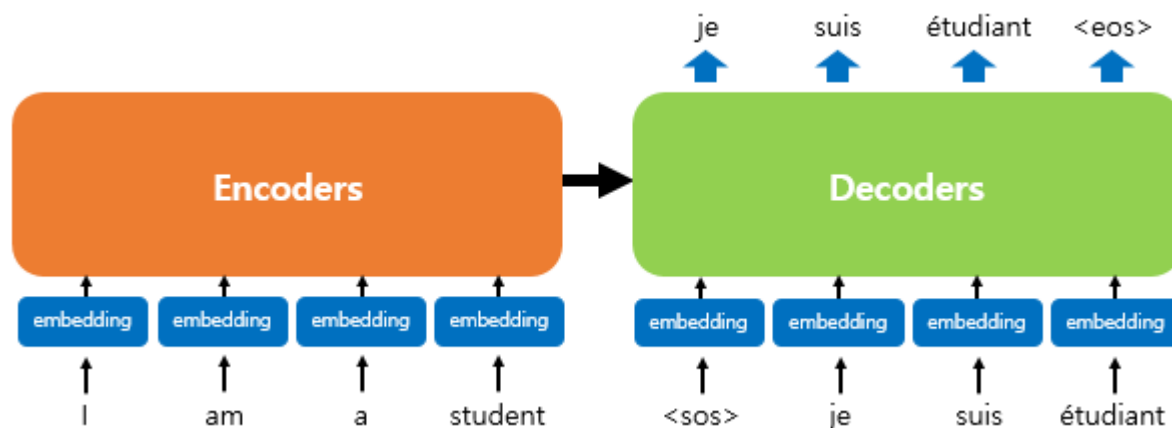
3. 트랜스포머(Transformer)



트랜스포머는 RNN을 사용하지 않지만 기존의 seq2seq처럼 인코더에서 입력 시퀀스를 입력받고, 디코더에서 출력 시퀀스를 출력하는 인코더-디코더 구조를 유지하고 있습니다. 이전 seq2seq 구조에서는 인코더와 디코더에서 각각 하나의 RNN이 t 개의 시점(time step)을 가지는 구조였다면 이번에는 인코더와 디코더라는 단위가 N 개로 구성되는 구조입니다. 트랜스포머를 제안한 논문에서는 인코더와 디코더의 개수를 각각 6개 사용하였습니다.



위의 그림은 인코더와 디코더가 6개씩 존재하는 트랜스포머의 구조를 보여줍니다. 이 책에서는 인코더와 디코더가 각각 여러 개 쌓여있다는 의미를 사용할 때는 알파벳 s를 뒤에 붙여 encoders, decoders라고 표현하겠습니다.



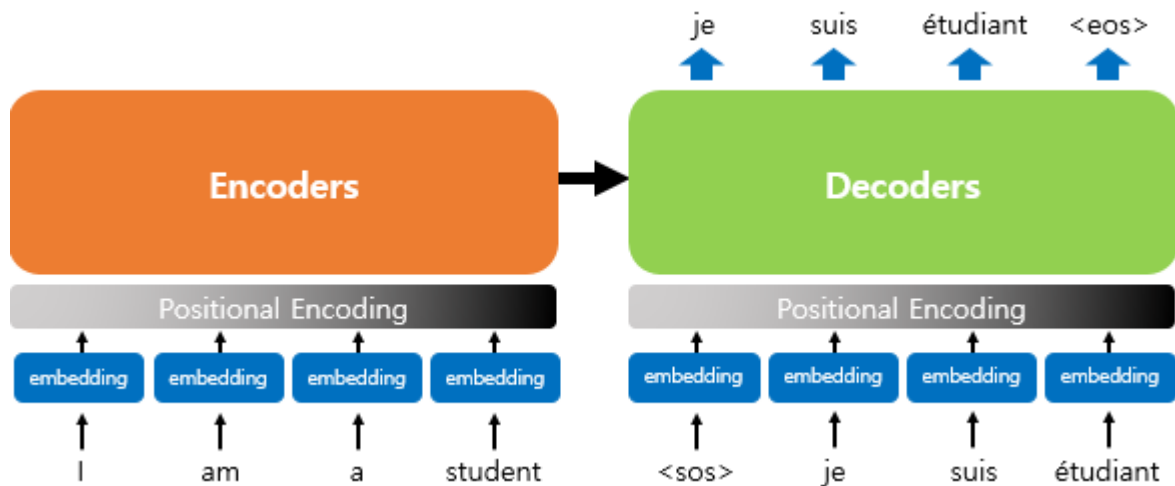
위의 그림은 인코더로부터 정보를 전달받아 디코더가 출력 결과를 만들어내는 트랜스포머 구조를 보여줍니다. 디코더는 마치 기존의 seq2seq 구조처럼 시작 심볼 <sos>를 입력으로 받아 종료 심볼 <eos>가 나올 때까지 연산을 진행합니다. 이는 RNN은 사용되지 않지만 여전히 인코더-디코더의 구조는 유지되고 있음을 보여줍니다.

트랜스포머의 내부 구조를 조금씩 확대해가는 방식으로 트랜스포머를 이해해보시다. 우선 인코더와 디코더의 구조를 이해하기 전에 트랜스포머의 입력에 대해서 이해해보겠습니다. 트랜스포머의 인코더와 디코더는 단순히 각 단어의 임베딩 벡터들을 입력받는 것이 아니라 임베딩 벡터에서 조정된 값을 입력받는데 이에 대해서 알아보기 위해 입력 부분을 확대해보겠습니다.

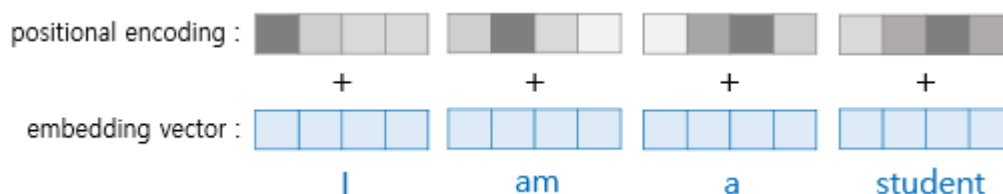
4. 포지셔널 인코딩(Positional Encoding)

트랜스포머의 내부를 이해하기 전 우선 **트랜스포머의 입력**에 대해서 알아보겠습니다. RNN이 자연어 처리에서 유용했던 이유는 단어의 위치에 따라 단어를 순차적으로 입력받아서 처리하는 RNN의 특성으로 인해 각 단어의 위치 정보(position information)를 가질 수 있다는 점에 있었습니다.

하지만 트랜스포머는 단어 입력을 순차적으로 받는 방식이 아니므로 단어의 위치 정보를 다른 방식으로 알려줄 필요가 있습니다. 트랜스포머는 단어의 위치 정보를 얻기 위해서 각 단어의 임베딩 벡터에 위치 정보들을 더하여 모델의 입력으로 사용하는데, 이를 포지셔널 인코딩(positional encoding)이라고 합니다.



위의 그림은 입력으로 사용되는 임베딩 벡터들이 트랜스포머의 입력으로 사용되기 전에 포지셔널 인코딩의 값이 더해지는 것을 보여줍니다. 임베딩 벡터가 인코더의 입력으로 사용되기 전 포지셔널 인코딩값이 더해지는 과정을 시각화하면 아래와 같습니다.



포지셔널 인코딩 값들은 어떤 값이기에 위치 정보를 반영해줄 수 있는 것일까요? 트랜스포머는 위치 정보를 가진 값을 만들기 위해서 아래의 두 개의 함수를 사용합니다.

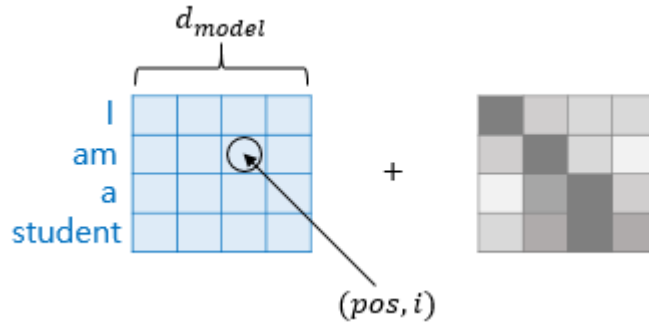
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

사인 함수와 코사인 함수의 그래프를 상기해보면 요동치는 값의 형태를 생각해볼 수 있는데, 트랜스포머는 사인 함수와 코사인 함수의 값을 임베딩 벡터에 더해주므로써 단어의 순서 정보를 더하여 줍니다. 그런데 위의 두 함수에는

pos, i, d_{model}

등의 생소한 변수들이 있습니다. 위의 함수를 이해하기 위해서는 위에서 본 임베딩 벡터와 포지셔널 인코딩의 덧셈은 사실 임베딩 벡터가 모여 만들어진 문장 행렬과 포지셔널 인코딩 행렬의 덧셈 연산을 통해 이루어진다는 점을 이해해야 합니다.



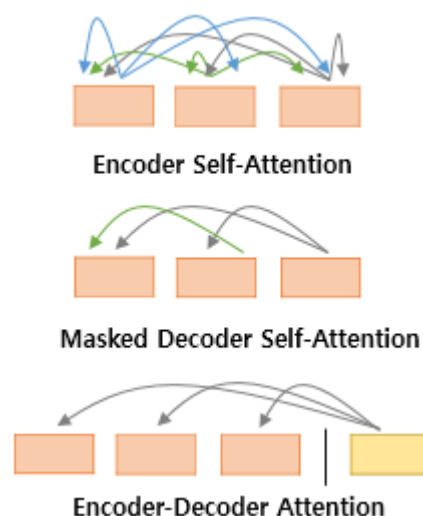
pos 는 입력 문장에서의 임베딩 벡터의 위치를 나타내며, i 는 임베딩 벡터 내의 차원의 인덱스를 의미합니다. 위의 식에 따르면 임베딩 벡터 내의 각 차원의 인덱스가 짝수인 경우에는 사인 함수의 값을 사용하고 홀수인 경우에는 코사인 함수의 값을 사용합니다. 위의 수식에서 $(pos, 2i)$ 일 때는 사인 함수를 사용하고, $(pos, 2i+1)$ 일 때는 코사인 함수를 사용하고 있음을 주목합니다.

또한 위의 식에서 d_{model} 은 트랜스포머의 모든 층의 출력 차원을 의미하는 트랜스포머의 하이퍼파라미터입니다. 앞으로 보게 될 트랜스포머의 각종 구조에서 d_{model} 의 값이 계속해서 등장하는 이유입니다. 임베딩 벡터 또한 d_{model} 의 차원을 가지는데 위의 그림에서는 마치 4로 표현되었지만 실제 논문에서는 512의 값을 가집니다.

위와 같은 포지셔널 인코딩 방법을 사용하면 순서 정보가 보존되는데, 예를 들어 각 임베딩 벡터에 포지셔널 인코딩의 값을 더하면 같은 단어라고 하더라도 문장 내의 위치에 따라서 트랜스포머의 입력으로 들어가는 임베딩 벡터의 값이 달라집니다. 이에 따라 트랜스포머의 입력은 순서 정보가 고려된 임베딩 벡터가 됩니다.

5. 어텐션(Attention)

트랜스포머에서 사용되는 세 가지의 어텐션에 대해서 간단히 정리해봅시다.



번째 그림인 셀프 어텐션은 인코더에서 이루어지지만, 두번째 그림인 셀프 어텐션과 세번째 그림인 인코더-디코더 어텐션은 디코더에서 이루어집니다. 셀프 어텐션은 본질적으로 Query, Key, Value가 동일한 경우를 말합니다. 반면, 세번째 그림 인코더-디코더 어텐션에서는 Query가 디코더의 벡터인 반면에 Key와 Value가 인코더의 벡터이므로 셀프 어텐션이라고 부르지 않습니다.

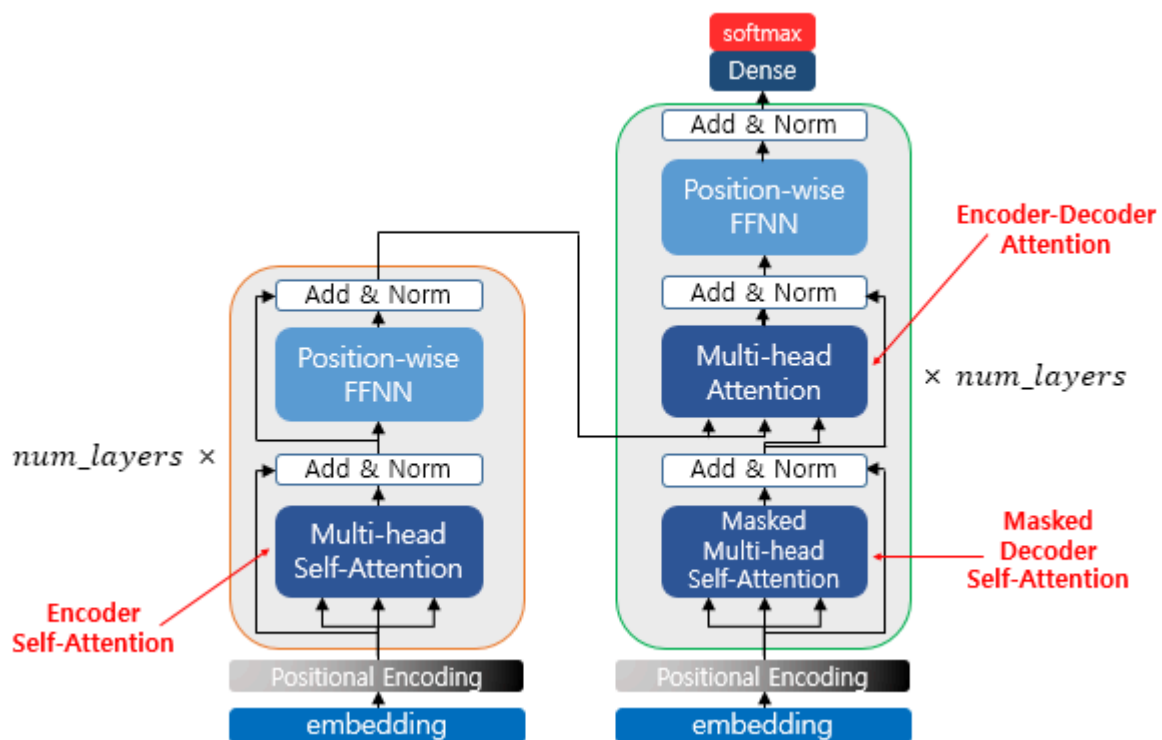
- 주의할 점은 여기서 Query, Key 등이 같다는 것은 벡터의 값이 같다는 것이 아니라 벡터의 출처가 같다는 의미입니다.

정리하면 다음과 같습니다.

인코더의 셀프 어텐션 : Query = Key = Value

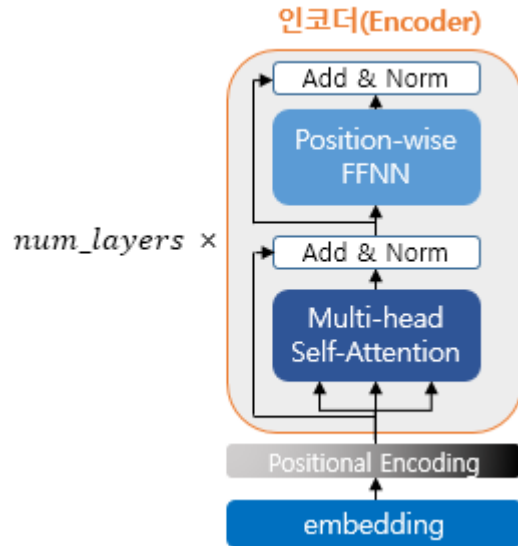
디코더의 마스크드 셀프 어텐션 : Query = Key = Value

디코더의 인코더-디코더 어텐션 : Query : 디코더 벡터 / Key = Value : 인코더 벡터



위 그림은 트랜스포머의 아키텍처에서 세 가지 어텐션이 각각 어디에서 이루어지는지를 보여줍니다. 세 개의 어텐션에 추가적으로 '멀티 헤드'라는 이름이 붙어있습니다. 뒤에서 설명하겠지만 트랜스포머가 어텐션을 병렬적으로 수행하는 방법을 의미합니다.

6. 인코더(Encoder)



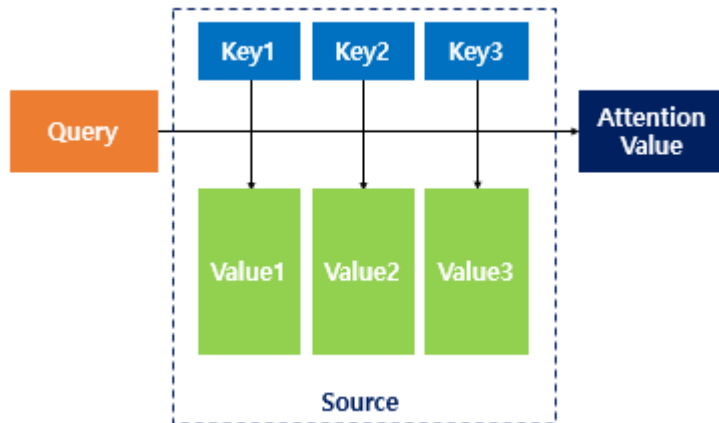
트랜스포머는 하이퍼파라미터인 `num_layers` 개수의 인코더 층을 쌓습니다. 논문에서는 총 6개의 인코더 층을 사용하였습니다. 인코더를 하나의 층이라는 개념으로 생각한다면, 하나의 인코더 층은 크게 총 2개의 서브층(sublayer)으로 나뉘어집니다. 셀프 어텐션과 피드 포워드 신경망입니다. 위의 그림에서는 멀티 헤드 셀프 어텐션과 포지션 와이즈 피드 포워드 신경망이라고 적혀있지만, 멀티 헤드 셀프 어텐션은 셀프 어텐션을 병렬적으로 사용하였다는 의미고, 포지션 와이즈 피드 포워드 신경망은 우리가 알고있는 일반적인 피드 포워드 신경망입니다. 우선 셀프 어텐션에 대해서 알아보시다.

7. 인코더의 셀프 어텐션

트랜스포머에서는 셀프 어텐션이라는 어텐션 기법이 등장하는데 앞서 배웠던 어텐션 함수에 대해서 복습하고, 셀프 어텐션이 앞서 배웠던 어텐션과 무엇이 다른지 이해해보겠습니다.

1) 셀프 어텐션의 의미와 이점

어텐션 함수는 주어진 '쿼리(Query)'에 대해서 모든 '키(Key)'와의 유사도를 각각 구합니다. 그리고 구해진 이 유사도를 가중치로 하여 키와 맵핑되어있는 각각의 '값(Value)'에 반영해줍니다. 그리고 유사도가 반영된 '값(Value)'을 모두 가중합하여 리턴합니다.



여기까지는 앞서 배운 어텐션의 개념입니다. 그런데 어텐션 중에서는 셀프 어텐션(self-attention)이라는 것이 있습니다. 어텐션을 자기 자신에게 수행한다는 의미입니다. 앞서 배운 seq2seq에서 어텐션을 사용할 경우의 Q, K, V의 정의를 다시 생각해봅시다.

사실 t 시점이라는 것은 계속 변화하면서 반복적으로 쿼리를 수행하므로 결국 전체 시점에 대해서 일반화를 할 수도 있습니다.

Q = Querys : 모든 시점의 디코더 셀에서의 은닉 상태들

K = Keys : 모든 시점의 인코더 셀의 은닉 상태들

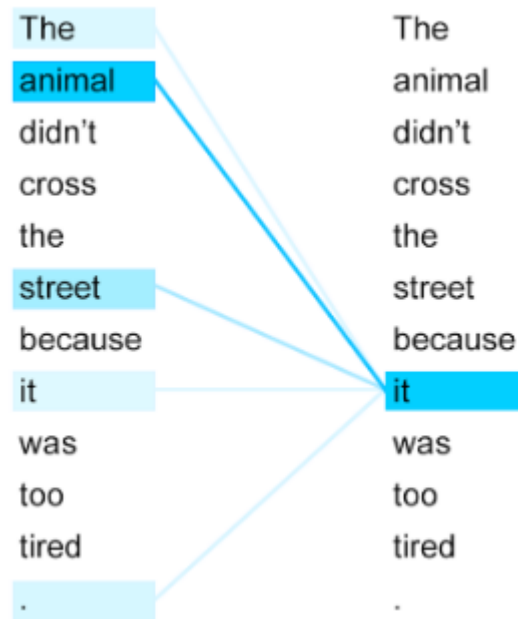
V = Values : 모든 시점의 인코더 셀의 은닉 상태들

이처럼 기존에는 디코더 셀의 은닉 상태가 Q이고 인코더 셀의 은닉 상태가 K라는 점에서 Q와 K가 서로 다른 값을 가지고 있었습니다. 그런데 셀프 어텐션에서는 Q, K, V가 전부 동일합니다. 트랜스포머의 셀프 어텐션에서의 Q, K, V는 아래와 같습니다.

Q : 입력 문장의 모든 단어 벡터들

K : 입력 문장의 모든 단어 벡터들

V : 입력 문장의 모든 단어 벡터들

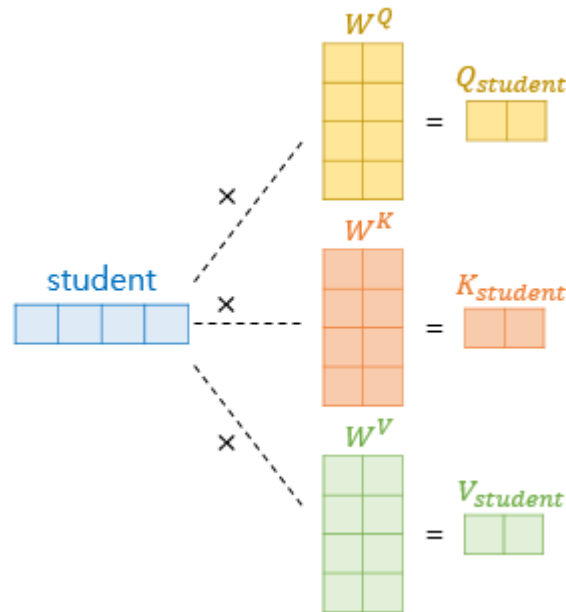


위의 그림은 트랜스포머에 대한 구글 AI 블로그 포스트에서 가져왔습니다. 위의 예시 문장을 번역하면 '그 동물은 길을 건너지 않았다. 왜냐하면 그것은 너무 피곤하였기 때문이다.' 라는 의미가 됩니다. 그런데 여기서 그것(it)에 해당하는 것은 과연 길(street)일까요? 동물(animal)일까요? 우리는 피곤한 주체가 동물이라는 것을 아주 쉽게 알 수 있지만 기계는 그렇지 않습니다. 하지만 셀프 어텐션은 입력 문장 내의 단어들끼리 유사도를 구하므로써 그것(it)이 동물(animal)과 연관되었을 확률이 높다는 것을 찾아냅니다.

2) Q, K, V 벡터 얻기

앞서 셀프 어텐션은 입력 문장의 단어 벡터들을 가지고 수행한다고 하였는데, 사실 셀프 어텐션은 인코더의 초기 입력인 d_{model} 의 차원을 가지는 단어 벡터들을 사용하여 셀프 어텐션을 수행하는 것이 아니라 우선 각 단어 벡터들로부터 Q벡터, K벡터, V벡터를 얻는 작업을 거칩니다. 이때 이 Q벡터, K벡터, V벡터들은 초기 입력인 d_{model} 의 차원을 가지는 단어 벡터들보다 더 작은 차원을 가지는데, 논문에서는 $d_{model}=512$ 의 차원을 가졌던 각 단어 벡터들을 64의 차원을 가지는 Q벡터, K벡터, V벡터로 변환하였습니다.

64라는 값은 트랜스포머의 또 다른 하이퍼파라미터인 num_heads 로 인해 결정되는데, 트랜스포머는 d_{model} 을 num_heads 로 나눈 값을 각 Q벡터, K벡터, V벡터의 차원으로 결정합니다. 논문에서는 num_heads 를 8로하였습니다. 그림을 통해 이해해봅시다. 예를 들어 여기서 사용하고 있는 예문 중 student라는 단어 벡터를 Q, K, V의 벡터로 변환하는 과정을 보겠습니다.



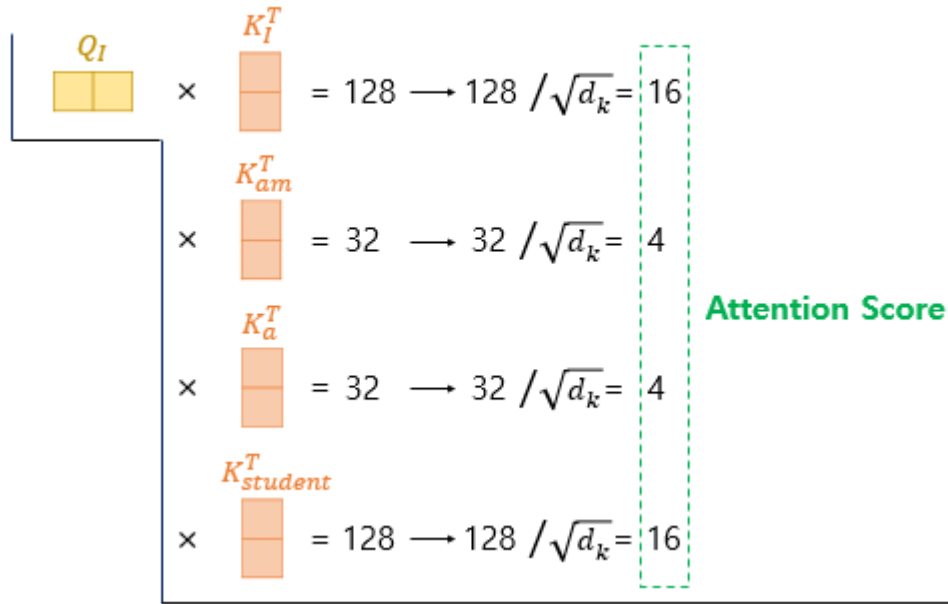
기존의 벡터로부터 더 작은 벡터는 가중치 행렬을 곱함으로써 완성됩니다. 각 가중치 행렬은 $d_{model} \times (d_{model}/num_heads)$ 의 크기를 가집니다. 이 가중치 행렬은 훈련 과정에서 학습됩니다. 즉, 논문과 같이 $d_{model}=512$ 이고 $num_heads=8$ 라면, 각 벡터에 3개의 서로 다른 가중치 행렬을 곱하고 64의 크기를 가지는 Q, K, V 벡터를 얻어냅니다. 위의 그림은 단어 벡터 중 student 벡터로부터 Q, K, V 벡터를 얻어내는 모습을 보여줍니다. 모든 단어 벡터에 위와 같은 과정을 거치면 I, am, a, student는 각각의 Q, K, V 벡터를 얻습니다.

3) 스케일드 닷-프로덕트 어텐션(Scaled dot-product Attention)

Q, K, V 벡터를 얻었다면 지금부터는 기존에 배운 어텐션 메커니즘과 동일합니다. 각 Q벡터는 모든 K벡터에 대해서 어텐션 스코어를 구하고, 어텐션 분포를 구한 뒤에 이를 사용하여 모든 V벡터를 가중합하여 어텐션 값 또는 컨텍스트 벡터를 구하게 됩니다. 그리고 이를 모든 Q벡터에 대해서 반복합니다.

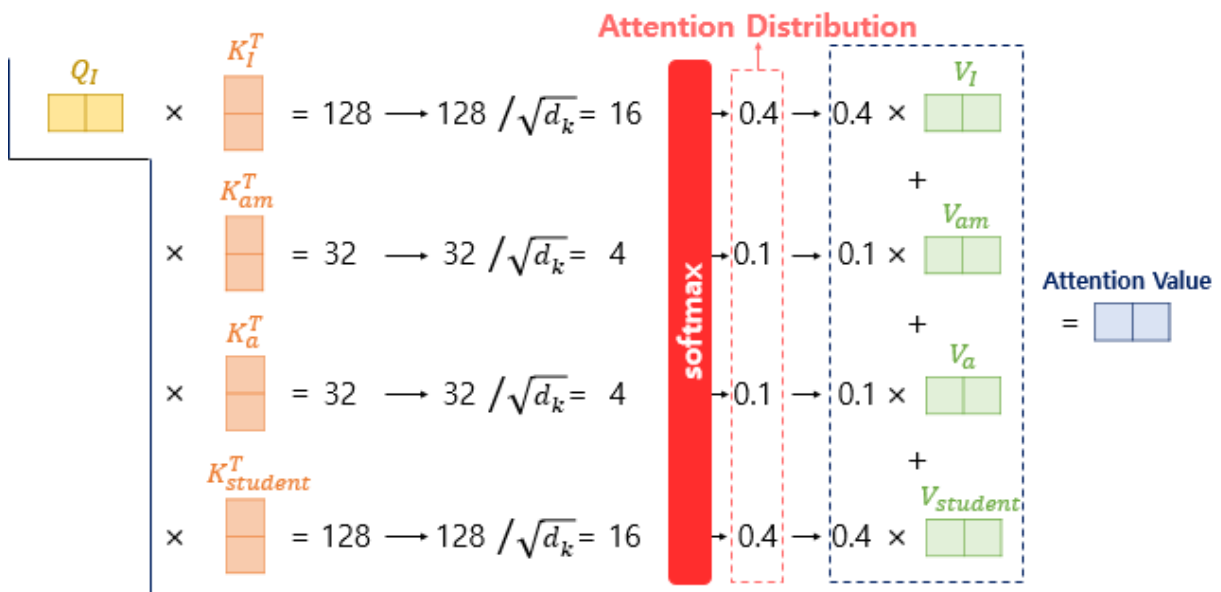
그런데 앞서 어텐션 챕터에서 어텐션 함수의 종류는 다양하다고 언급한 바 있습니다. 트랜스포머에서는 어텐션 챕터에 사용했던 내적만을 사용하는 어텐션 함수 $score(q,k)=q \cdot k$ 가 아니라 여기에 특정값으로 나눠준 어텐션 함수인 $score(q,k)=q \cdot k/n$ 를 사용합니다. 이러한 함수를 사용하는 어텐션을 어텐션 챕터에서 배운 닷-프로덕트 어텐션(dot-product attention)에서 값을 스케일링하는 것을 추가하였다고 하여 **스케일드 닷-프로덕트 어텐션(Scaled dot-product Attention)**이라고 합니다. 그림을 통해 이해해봅시다.

Scaled dot product Attention : $score\ function(q, k) = q \cdot k / \sqrt{n}$



우선 단어 I에 대한 Q벡터를 기준으로 설명해보겠습니다. 지금부터 설명하는 과정은 am에 대한 Q벡터, a에 대한 Q벡터, student에 대한 Q벡터에 대해서도 모두 동일한 과정을 거칩니다. 위의 그림은 단어 I에 대한 Q벡터가 모든 K벡터에 대해서 어텐션 스코어를 구하는 것을 보여줍니다. 위의 128과 32는 저자가 임의로 가정한 수치로 신경쓰지 않아도 좋습니다.

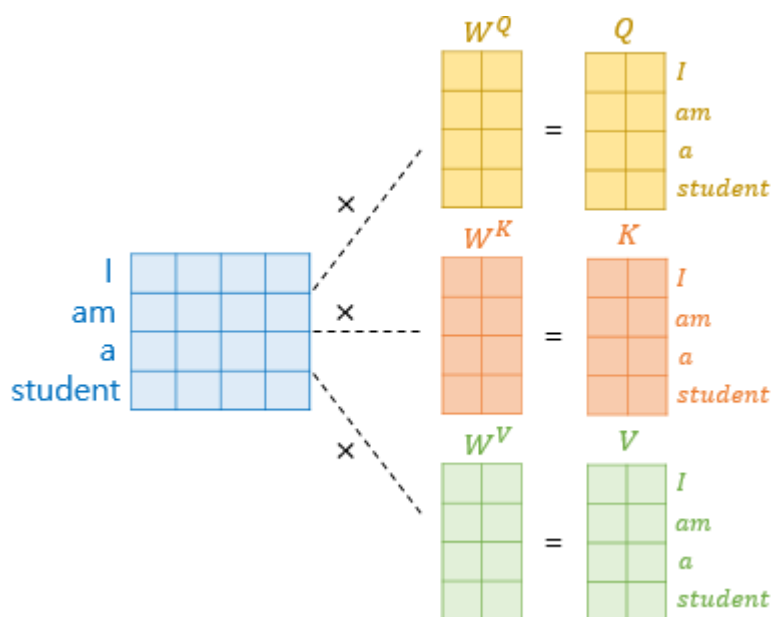
위의 그림에서 어텐션 스코어는 각각 단어 I가 단어 I, am, a, student와 얼마나 연관되어 있는지를 보여주는 수치입니다. 트랜스포머에서는 두 벡터의 내적값을 스케일링하는 값으로 K 벡터의 차원을 나타내는 d_k 에 루트를 씌운 $\sqrt{d_k}$ 사용하는 것을 택했습니다. 앞서 언급하였듯이 논문에서 d_k 는 d_{model}/num_heads 라는 식에 따라서 64의 값을 가지므로 d_k 는 8의 값을 가집니다.



이제 어텐션 스코어에 소프트맥스 함수를 사용하여 어텐션 분포(Attention Distribution)를 구하고, 각 V벡터와 가중합하여 어텐션 값(Attention Value)을 구합니다. 이를 단어 I에 대한 어텐션 값 또는 단어 I에 대한 컨텍스트 벡터(context vector)라고도 할 수 있습니다. am에 대한 Q벡터, a에 대한 Q벡터, student에 대한 Q벡터에 대해서도 모두 동일한 과정을 반복하여 각각에 대한 어텐션 값을 구합니다. 그런데 한 가지 의문이 남습니다. 굳이 이렇게 각 Q벡터마다 일일이 따로 연산할 필요가 있을까요?

4) 행렬 연산으로 일괄 처리하기

사실 각 단어에 대한 Q, K, V 벡터를 구하고 스케일드 닷-프로덕트 어텐션을 수행하였던 위의 과정들은 벡터 연산이 아니라 행렬 연산을 사용하면 일괄 계산이 가능합니다. 지금까지 벡터 연산으로 설명하였던 이유는 이해를 돕기 위한 과정이고, 실제로는 행렬 연산으로 구현됩니다. 위의 과정을 벡터가 아닌 행렬 연산으로 이해해봅시다. 우선, 각 단어 벡터마다 일일이 가중치 행렬을 곱하는 것이 아니라 문장 행렬에 가중치 행렬을 곱하여 Q행렬, K행렬, V행렬을 구합니다.



행렬 연산을 통해 어텐션 스코어는 어떻게 구할 수 있을까요? 여기서 Q행렬을 K행렬을 전치한 행렬과 곱해준다고 해봅시다. 이렇게 되면 각각의 단어의 Q벡터와 K벡터의 내적이 각 행렬의 원소가 되는 행렬이 결과로 나옵니다.

$$\begin{array}{c} Q \\ \begin{array}{c} I \\ am \\ a \\ student \end{array} \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{array} \times \begin{array}{c} I \ am \ a \ student \\ K^T \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{array} = \begin{array}{c} I \ am \ a \ student \\ \begin{array}{c} I \\ am \\ a \\ student \end{array} \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{array}$$

다시 말해 위의 그림의 결과 행렬의 값에 전체적으로 d_k 를 나누어주면 이는 각 행과 열이 어텐션 스코어 값을 가지는 행렬이 됩니다. 예를 들어 I 행과 student 열의 값은 I의 Q벡터와 student의 K벡터의 어텐션 스코어 값입니다. 위 행렬을 어텐션 스코어 행렬이라 합시다. 어텐션 스코어 행렬을 구하였다면 남은 것은 어텐션 분포를 구하고, 이를 사용하여 모든 단어에 대한 어텐션 값을 구하는 일입니다. 이는 간단하게 어텐션 스코어 행렬에 소프트맥스 함수를 사용하고, V행렬을 곱하는 것으로 해결됩니다. 이렇게 되면 각 단어의 어텐션 값을 모두 가지는 어텐션 값 행렬이 결과로 나옵니다.

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V = \text{Attention Value Matrix } a$$

위의 그림은 행렬 연산을 통해 모든 값이 일괄 계산되는 과정을 식으로 보여줍니다. 해당 식은 실제 트랜스포머 논문에 기재된 아래의 수식과 정확하게 일치하는 식입니다.

$$\text{Attention}(Q,K,V)=\text{softmax}(QK^Tdk)V$$

위의 행렬 연산에 사용된 행렬의 크기를 모두 정리해봅시다. 우선 입력 문장의 길이를 seq_len이라고 해봅시다. 그렇다면 문장 행렬의 크기는 (seq_len, dmodel)입니다. 여기에 3개의 가중치 행렬을 곱해서 Q, K, V 행렬을 만들어야 합니다.

우선 행렬의 크기를 정의하기 위해 행렬의 각 행에 해당되는 Q벡터와 K벡터의 차원을 d_k 라고 하고, V벡터의 차원을 d_v 라고 해봅시다. 그렇다면 Q행렬과 K행렬의 크기는 (seq_len, d_k)이며, V행렬의 크기는 (seq_len, d_v)가 되어야 합니다. 그렇다면 문장 행렬과 Q, K, V 행렬의 크기로부터 가중치 행렬의 크기 추정이 가능합니다. WQ와 WK는 (dmodel, d_k)의 크기를 가지며, WV는 (dmodel, d_v)의 크기를 가집니다. 단, 논문에서는 d_k 와 d_v 의 차원은 $d_{\text{model}}/\text{num_heads}$ 와 같습니다.

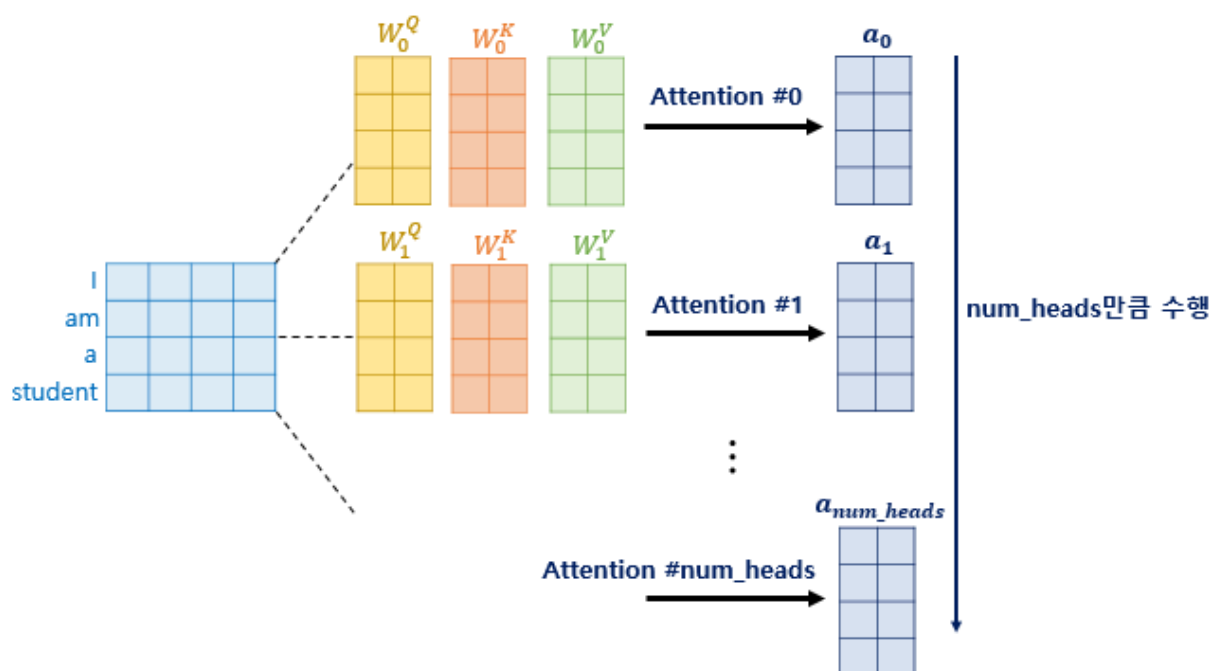
즉, $d_{\text{model}}/\text{num_heads}=d_k=d_v$ 입니다.

결과적으로 $\text{softmax}(QK^Tdk)V$ 식을 적용하여 나오는 어텐션 값 행렬 a의 크기는 (seq_len, d_v)이 됩니다.

6) 멀티 헤드 어텐션(Multi-head Attention)

앞서 배운 어텐션에서는 dmodel의 차원을 가진 단어 벡터를 num_heads로 나눈 차원을 가지는 Q, K, V벡터로 바꾸고 어텐션을 수행하였습니다. 논문 기준으로는 512의 차원의 각 단어 벡터를 8로 나누어 64차원의 Q, K, V 벡터로 바꾸어서 어텐션을 수행한 셈인데, 이

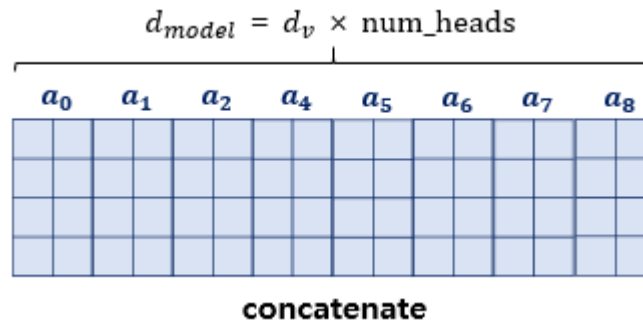
제 `num_heads`의 의미와 왜 `dmodel`의 차원을 가진 단어 벡터를 가지고 어텐션을 하지 않고 차원을 축소시킨 벡터로 어텐션을 수행하였을까.



트랜스포머 연구진은 한 번의 어텐션을 하는 것보다 여러번의 어텐션을 병렬로 사용하는 것이 더 효과적이라고 판단하였습니다. 그래서 `dmodel`의 차원을 `num_heads`개로 나누어 `dmodel/num_heads`의 차원을 가지는 Q, K, V에 대해서 `num_heads`개의 병렬 어텐션을 수행합니다. 논문에서는 하이퍼파라미터인 `num_heads`의 값을 8로 지정하였고, 8개의 병렬 어텐션이 이루어지게 됩니다. 다시 말해 위에서 설명한 어텐션이 8개로 병렬로 이루어지게 되는데, 이때 각각의 어텐션 값 행렬을 어텐션 헤드라고 부릅니다. 이때 가중치 행렬 W^Q, W^K, W^V 의 값은 8개의 어텐션 헤드마다 전부 다릅니다.

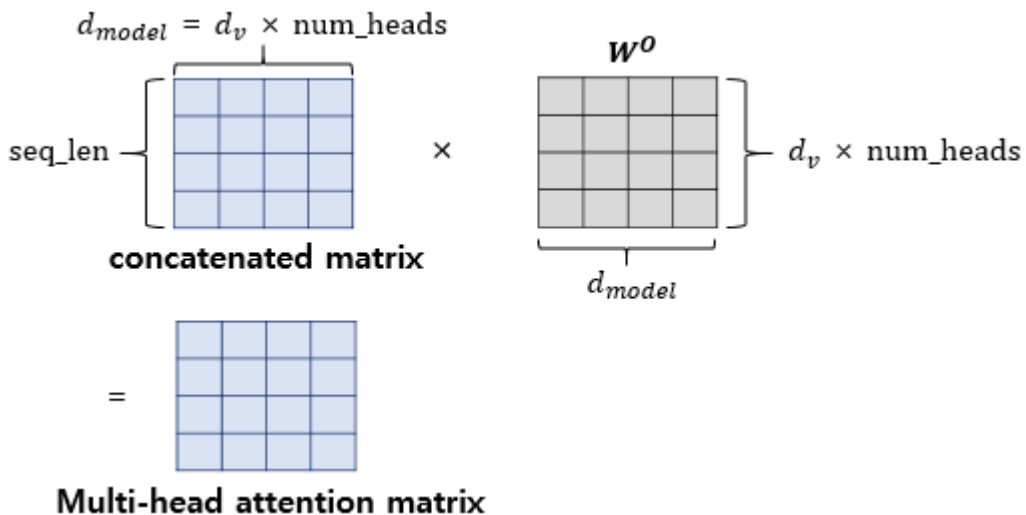
병렬 어텐션으로 얻을 수 있는 효과는 무엇일까요? 그리스로마신화에는 머리가 여러 개인 괴물 히드라나 케로베로스가 나옵니다. 이 괴물들의 특징은 머리가 여러 개이기 때문에 여러 시점에서 상대방을 볼 수 있다는 겁니다. 이렇게 되면 시각에서 놓치는 게 별로 없을테니까 이런 괴물들에게 기습을 하는 것이 굉장히 힘이 듭니다. 멀티 헤드 어텐션도 똑같습니다. 어텐션을 병렬로 수행하여 다른 시각으로 정보들을 수집하겠다는 겁니다.

예를 들어보겠습니다. 앞서 사용한 예문 '그 동물은 길을 건너지 않았다. 왜냐하면 그것은 너무 피곤하였기 때문이다.'를 상기해봅시다. 단어 그것(it)이 쿼리였다고 해봅시다. 즉, it에 대한 Q벡터로부터 다른 단어와의 연관도를 구하였을 때 첫번째 어텐션 헤드는 '그것(it)'과 '동물(animal)'의 연관도를 높게 본다면, 두번째 어텐션 헤드는 '그것(it)'과 '피곤하였기 때문이다(tired)'의 연관도를 높게 볼 수 있습니다. 각 어텐션 헤드는 전부 다른 시각에서 보고있기 때문입니다.



병렬 어텐션을 모두 수행하였다면 모든 어텐션 헤드를 연결(concatenate)합니다. 모두 연결된 어텐션 헤드 행렬의 크기는 (seq_len, dmodel)가 됩니다.

지금까지 그림에서는 책의 지면상의 한계로 4차원을 dmodel=512로 표현하고, 2차원을 $d_v=64$ 로 표현해왔기 때문에 위의 그림의 행렬의 크기에 혼동의 있을 수 있으나 8개의 어텐션 헤드의 연결(concatenate) 과정의 이해를 위해 이번 행렬만 예외로 위와 같이 dmodel의 크기를 d_v 의 8배인 16차원으로 표현하였습니다. 아래의 그림에서는 다시 dmodel를 4차원으로 표현합니다.



어텐션 헤드를 모두 연결한 행렬은 또 다른 가중치 행렬 W^0 을 곱하게 되는데, 이렇게 나온 결과 행렬이 멀티-헤드 어텐션의 최종 결과물입니다. 위의 그림은 어텐션 헤드를 모두 연결한 행렬이 가중치 행렬 W^0 과 곱해지는 과정을 보여줍니다. 이때 결과물인 멀티-헤드 어텐션 행렬은 인코더의 입력이었던 문장 행렬의 (seq_len, dmodel) 크기와 동일합니다.

다시 말해 인코더의 첫번째 서브층인 멀티-헤드 어텐션 단계를 끝마쳤을 때, 인코더의 입력으로 들어왔던 행렬의 크기가 아직 유지되고 있음을 기억해둡시다. 첫번째 서브층인 멀티-헤드 어텐션과 두번째 서브층인 포지션 와이즈 피드 포워드 신경망을 지나면서 인코더의 입력으로 들어올 때의 행렬의 크기는 계속 유지되어야 합니다. 트랜스포머는 동일한 구조의 인코더를 쌓은 구조입니다. 논문 기준으로는 인코더가 총 6개입니다. 인코더에서의 입력의 크기

가 출력에서도 동일 크기로 계속 유지되어야만 다음 인코더에서도 다시 입력이 될 수 있습니다.

8) 패딩 마스크(Padding Mask)

스케일드 닷 프로덕트 어텐션 함수 내부를 보면 mask라는 값을 인자로 받아서, 이 mask값에다가 -1e9라는 아주 작은 음수값을 곱한 후 어텐션 스코어 행렬에 더해주고 있습니다. 이 연산의 정체는 무엇일까요?

```
def scaled_dot_product_attention(query, key, value, mask):
    ... 중략 ...
    logits += (mask * -1e9) # 어텐션 스코어 행렬인 logits에 mask*-1e9 값을 더해
    주고 있다.
    ... 중략 ...
```

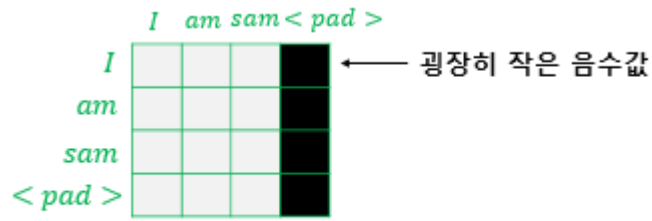
이는 입력 문장에 <PAD> 토큰이 있을 경우 어텐션에서 사실상 제외하기 위한 연산입니다. 예를 들어 <PAD>가 포함된 입력 문장의 셀프 어텐션의 예제를 봅시다. 이에 대해서 어텐션을 수행하고 어텐션 스코어 행렬을 얻는 과정은 다음과 같습니다.

$$\begin{array}{c} \text{Q} \\ \begin{array}{c} I \\ am \\ sam \\ <pad> \end{array} \end{array} \times \begin{array}{c} K^T \\ \begin{array}{c} I \quad am \quad sam \quad <pad> \end{array} \end{array} = \begin{array}{c} \begin{array}{c} I \quad am \quad sam \quad <pad> \end{array} \\ \begin{array}{c} I \\ am \\ sam \\ <pad> \end{array} \end{array}$$

$\sqrt{d_k}$

Attention Score Matrix

그런데 사실 단어 <PAD>의 경우에는 실질적인 의미를 가진 단어가 아닙니다. 그래서 트랜스포머에서는 Key의 경우에 <PAD> 토큰이 존재한다면 이에 대해서는 유사도를 구하지 않도록 마스킹(Masking)을 해주기로 했습니다. 여기서 마스킹이란 어텐션에서 제외하기 위해 값을 가린다는 의미입니다. 어텐션 스코어 행렬에서 행에 해당하는 문장은 Query이고, 열에 해당하는 문장은 Key입니다. 그리고 Key에 <PAD>가 있는 경우에는 해당 열 전체를 마스킹을 해줍니다.



Attention Score Matrix

마스킹을 하는 방법은 어텐션 스코어 행렬의 마스킹 위치에 매우 작은 음수값을 넣어주는 것입니다. 여기서 매우 작은 음수값이라는 것은 -1,000,000,000과 같은 -무한대에 가까운 수라는 의미입니다. 현재 어텐션 스코어 함수는 소프트맥스 함수를 지나지 않은 상태입니다. 앞서 배운 연산 순서라면 어텐션 스코어 함수는 소프트맥스 함수를 지나고, 그 후 Value 행렬과 곱해지게 됩니다. 그런데 현재 마스킹 위치에 매우 작은 음수 값이 들어가 있으므로 어텐션 스코어 행렬이 소프트맥스 함수를 지난 후에는 해당 위치의 값은 0이 되어 단어 간 유사도를 구하는 일에 <PAD> 토큰이 반영되지 않게 됩니다.



Attention Score Matrix

위 그림은 소프트맥스 함수를 지난 후를 가정하고 있습니다. 소프트맥스 함수를 지나면 각 행의 어텐션 가중치의 총 합은 1이 되는데, 단어 <PAD>의 경우에는 0이 되어 어떤 유의미한 값을 가지고 있지 않습니다.

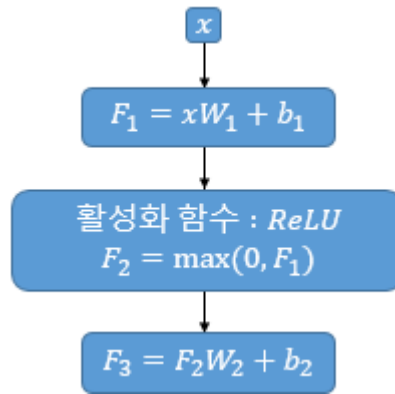
패딩 마스크를 구현하는 방법은 입력된 정수 시퀀스에서 패딩 토큰의 인덱스인지, 아닌지를 판별하는 함수를 구현하는 것입니다. 아래의 함수는 정수 시퀀스에서 0인 경우에는 1로 변환하고, 그렇지 않은 경우에는 0으로 변환하는 함수입니다.

포지션-와이즈 피드 포워드 신경망(Position-wise FFNN)

지금은 인코더를 설명하고 있지만, 포지션 와이즈 FFNN은 인코더와 디코더에서 공통적으로 가지고 있는 서브층입니다. 포지션-와이즈 FFNN는 쉽게 말하면 완전 연결 FFNN(Fully-connected FFNN)이라고 해석할 수 있습니다. 앞서 인공 신경망은 결국 벡터와 행렬 연산으로 표현될 수 있음을 배웠습니다. 아래는 포지션 와이즈 FFNN의 수식을 보여줍니다.

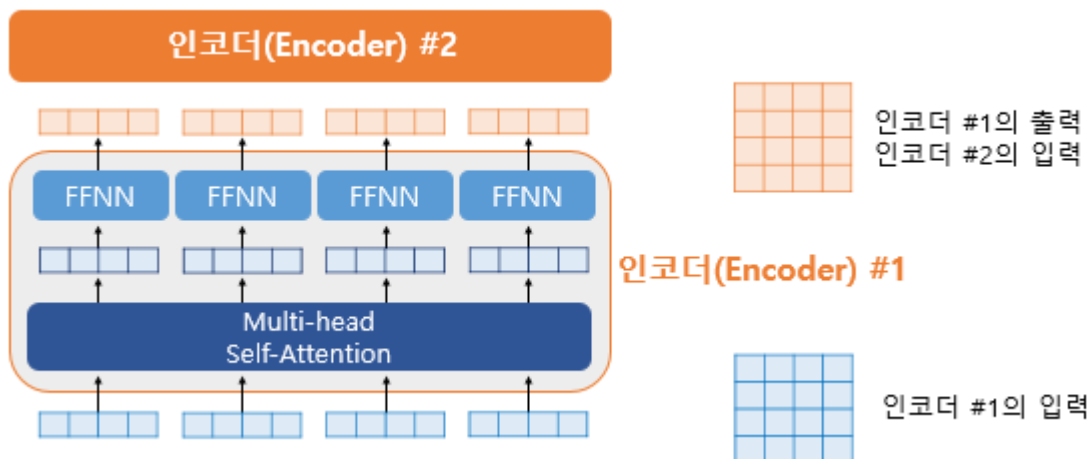
$$\text{FFNN}(x) = \text{MAX}(0, xW_1 + b_1)W_2 + b_2$$

식을 그림으로 표현하면 아래와 같습니다.



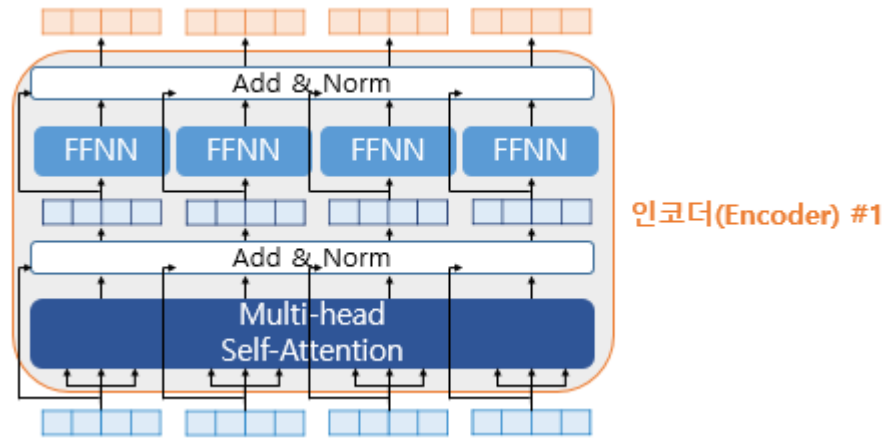
여기서 x 는 앞서 멀티 헤드 어텐션의 결과로 나온 $(\text{seq_len}, \text{dmodel})$ 의 크기를 가지는 행렬을 말합니다. 가중치 행렬 W_1 은 $(\text{dmodel}, \text{dff})$ 의 크기를 가지고, 가중치 행렬 W_2 은 $(\text{dff}, \text{dmodel})$ 의 크기를 가집니다. 논문에서 은닉층의 크기인 dff 는 앞서 하이퍼파라미터를 정의할 때 언급했듯이 2,048의 크기를 가집니다.

여기서 매개변수 W_1, b_1, W_2, b_2 는 하나의 인코더 층 내에서는 다른 문장, 다른 단어들마다 정확하게 동일하게 사용됩니다. 하지만 인코더 층마다 다른 값을 가집니다.



위의 그림에서 좌측은 인코더의 입력을 벡터 단위로 봤을 때, 각 벡터들이 멀티 헤드 어텐션 층이라는 인코더 내 첫번째 서브 층을 지나 FFNN을 통과하는 것을 보여줍니다. 이는 두번째 서브층인 Position-wise FFNN을 의미합니다. 물론, 실제로는 그림의 우측과 같이 행렬로 연산되는데, 두번째 서브층을 지난 인코더의 최종 출력은 여전히 인코더의 입력의 크기였던 $(\text{seq_len}, \text{dmodel})$ 의 크기가 보존되고 있습니다. 하나의 인코더 층을 지난 이 행렬은 다음 인코더 층으로 전달되고, 다음 층에서도 동일한 인코더 연산이 반복됩니다.

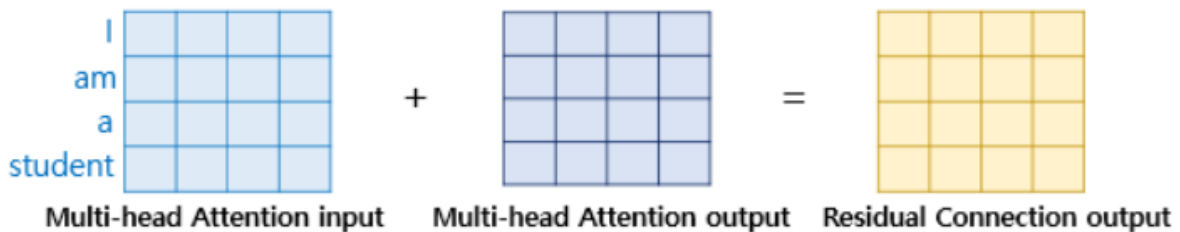
잔차 연결(Residual connection)과 층 정규화(Layer Normalization)



인코더의 두 개의 서브층에 대해서 이해하였다면 인코더에 대한 설명은 거의 끝났습니다! 트랜스포머에서는 이러한 두 개의 서브층을 가진 인코더에 추가적으로 사용하는 기법이 있는데, 바로 Add & Norm입니다. 더 정확히는 잔차 연결(residual connection)과 층 정규화(layer normalization)를 의미합니다.

가령, 서브층이 멀티 헤드 어텐션이었다면 잔차 연결 연산은 다음과 같습니다.

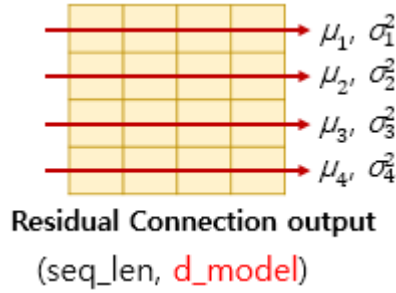
$$H(x) = x + \text{Multi-head Attention}(x)$$



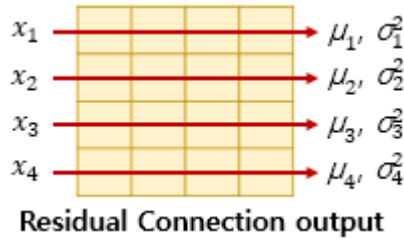
잔차 연결의 입력을 x , 잔차 연결과 층 정규화 두 가지 연산을 모두 수행한 후의 결과 행렬을 LN이라고 하였을 때, 잔차 연결 후 층 정규화 연산을 수식으로 표현하자면 다음과 같습니다.

$$LN = \text{LayerNorm}(x + \text{Sublayer}(x))$$

층 정규화는 **텐서의 마지막 차원**에 대해서 평균과 분산을 구하고, 이를 가지고 어떤 수식을 통해 값을 정규화하여 학습을 돕습니다. 여기서 텐서의 마지막 차원이란 것은 트랜스포머에서는 d_{model} 차원을 의미합니다. 아래 그림은 d_{model} 차원의 방향을 화살표로 표현하였습니다.



층 정규화를 위해서 우선, 화살표 방향으로 각각 평균 μ 과 분산 σ^2 을 구합니다. 각 화살표 방향의 벡터를 x_i 라고 명명해봅시다.



층 정규화를 수행한 후에는 벡터 x_i 는 ln_i 라는 벡터로 정규화가 됩니다.

$$ln_i = LayerNorm(x_i)$$

층 정규화의 수식을 알아보시다. 여기서는 층 정규화를 두 가지 과정으로 나누어서 설명하겠습니다. 첫번째는 평균과 분산을 통한 정규화, 두번째는 감마와 베타를 도입하는 것입니다. 우선, 평균과 분산을 통해 벡터 x_i 를 정규화 해줍니다. x_i 는 벡터인 반면, 평균 μ_i 과 분산 σ_i^2 은 스칼라입니다. 벡터 x_i 의 각 차원을 k 라고 하였을 때, $x_{i,k}$ 는 다음의 수식과 같이 정규화할 수 있습니다. 다시 말해 벡터 x_i 의 각 k 차원의 값이 다음과 같이 정규화 되는 것입니다.

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

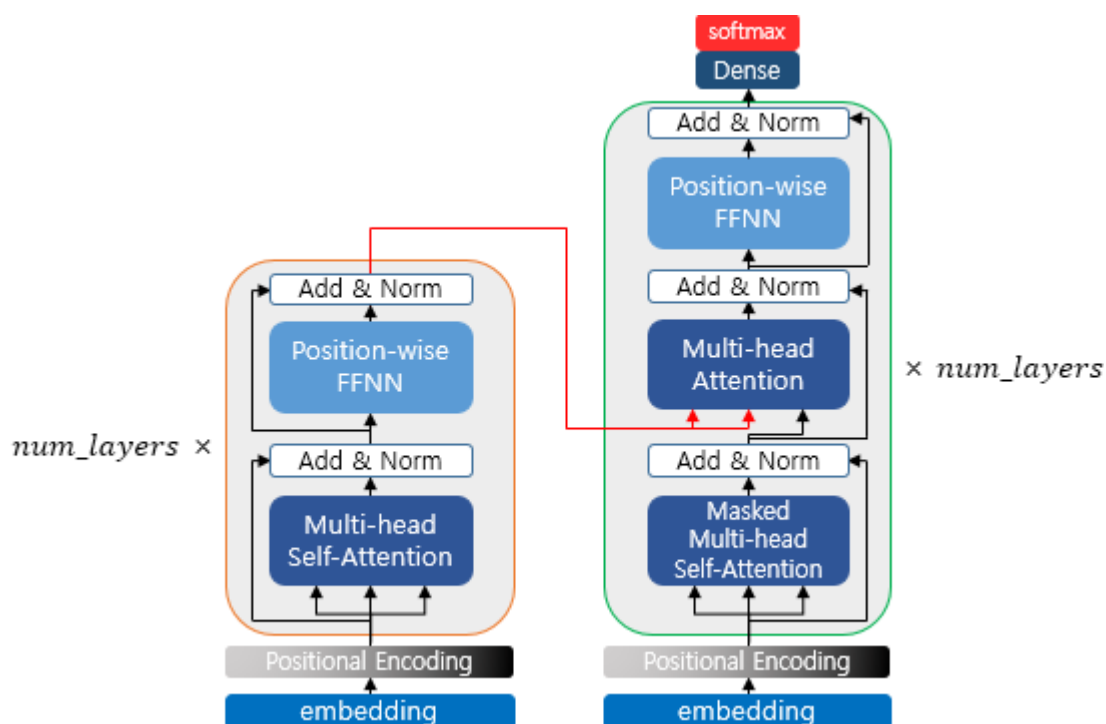
(입실론)은 분모가 0이 되는 것을 방지하는 값입니다.

이제 γ (감마)와 β (베타)라는 벡터를 준비합니다. 단, 이들의 초기값은 각각 1과 0입니다.

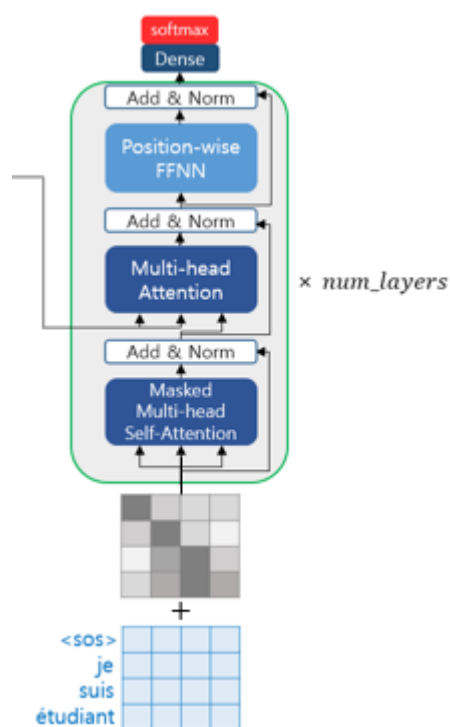
γ 와 β 를 도입한 층 정규화의 최종 수식은 다음과 같으며 γ 와 β 는 학습 가능한 파라미터입니다.

인코더에서 디코더로(From Encoder To Decoder)

$$ln_i = \gamma \hat{x}_i + \beta = LayerNorm(x_i)$$



디코더의 첫번째 서브층 : 셀프 어텐션과 룩-어헤드 마스크

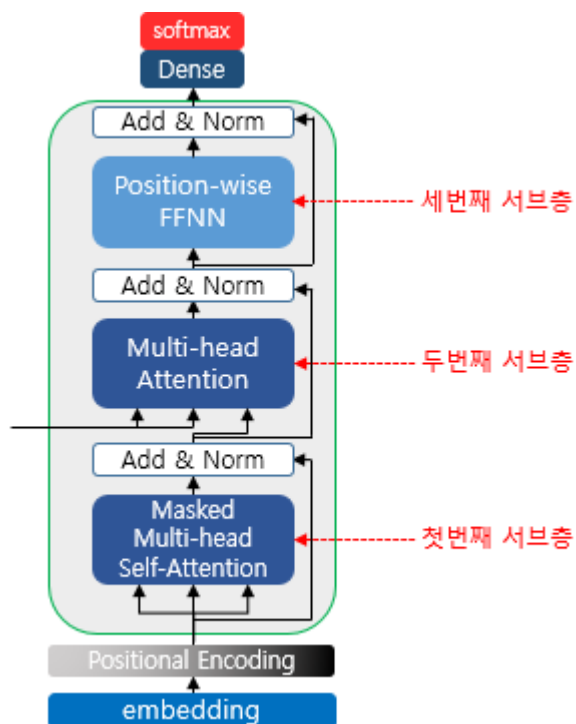


위 그림과 같이 디코더도 인코더와 동일하게 임베딩 층과 포지셔널 인코딩을 거친 후의 문장 행렬이 입력됩니다. 트랜스포머 또한 seq2seq와 마찬가지로 교사 강요(Teacher

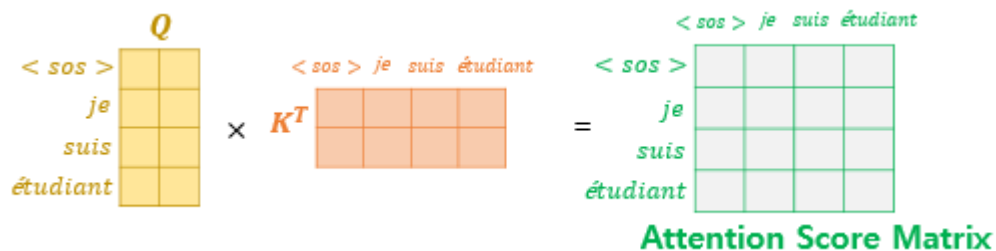
Forcing)을 사용하여 훈련되므로 학습 과정에서 디코더는 번역할 문장에 해당되는 `<sos> je suis étudiant` 의 문장 행렬을 한 번에 입력받습니다. 그리고 디코더는 이 문장 행렬로부터 각 시점의 단어를 예측하도록 훈련됩니다.

여기서 문제가 있습니다. seq2seq의 디코더에 사용되는 RNN 계열의 신경망은 입력 단어를 매 시점마다 순차적으로 입력받으므로 다음 단어 예측에 현재 시점을 포함한 이전 시점에 입력된 단어들만 참고할 수 있습니다. 반면, 트랜스포머는 문장 행렬로 입력을 한 번에 받으므로 현재 시점의 단어를 예측하고자 할 때, 입력 문장 행렬로부터 미래 시점의 단어까지도 참고할 수 있는 현상이 발생합니다. 가령, `suis`를 예측해야 하는 시점이라고 해봅시다. RNN 계열의 seq2seq의 디코더라면 현재까지 디코더에 입력된 단어는 `<sos>`와 `je`뿐일 것입니다. 반면, 트랜스포머는 이미 문장 행렬로 `<sos> je suis étudiant` 를 입력받았습니다.

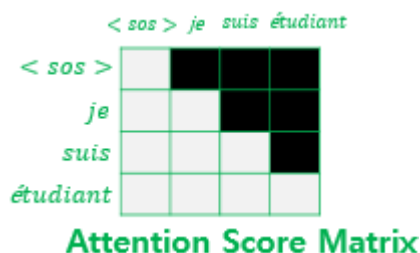
이를 위해 트랜스포머의 디코더에서는 현재 시점의 예측에서 현재 시점보다 미래에 있는 단어들을 참고하지 못하도록 룩-어헤드 마스크(look-ahead mask)를 도입했습니다. 직역하면 '미리보기에 대한 마스크'입니다.



룩-어헤드 마스크(look-ahead mask)는 디코더의 첫번째 서브층에서 이루어집니다. 디코더의 첫번째 서브층인 멀티 헤드 셀프 어텐션 층은 인코더의 첫번째 서브층인 멀티 헤드 셀프 어텐션 층과 동일한 연산을 수행합니다. 오직 다른 점은 어텐션 스코어 행렬에서 마스킹을 적용한다는 점만 다릅니다. 우선 다음과 같이 셀프 어텐션을 통해 어텐션 스코어 행렬을 얻습니다.



이제 자기 자신보다 미래에 있는 단어들은 참고하지 못하도록 다음과 같이 마스킹합니다.



마스킹 된 후의 어텐션 스코어 행렬의 각 행을 보면 자기 자신과 그 이전 단어들을만 참고할 수 있음을 볼 수 있습니다. 그 외에는 근본적으로 셀프 어텐션이라는 점과, 멀티 헤드 어텐션을 수행한다는 점에서 인코더의 첫번째 서브층과 같습니다.

룩-어헤드 마스크의 구현에 대해 알아보시다. 룩-어헤드 마스크는 패딩 마스크와 마찬가지로 앞서 구현한 스케일드 닷 프로덕트 어텐션 함수에 mask라는 인자로 전달됩니다. 패딩 마스킹을 써야하는 경우에는 스케일드 닷 프로덕트 어텐션 함수에 패딩 마스크를 전달하고, 룩-어헤드 마스킹을 써야하는 경우에는 스케일드 닷 프로덕트 어텐션 함수에 룩-어헤드 마스크를 전달합니다.

디코더의 두번째 서브층 : 인코더-디코더 어텐션

디코더의 두번째 서브층은 멀티 헤드 어텐션을 수행한다는 점에서는 이전의 어텐션들(인코더와 디코더의 첫번째 서브층)과는 공통점이 있으나 이번에는 셀프 어텐션이 아닙니다.

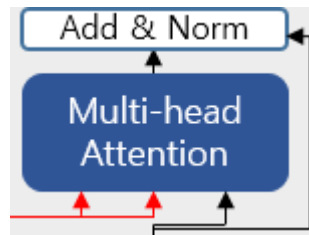
셀프 어텐션은 Query, Key, Value가 같은 경우를 말하는데, 인코더-디코더 어텐션은 Query가 디코더인 행렬인 반면, Key와 Value는 인코더 행렬이기 때문입니다. 다시 한 번 각 서브층에서의 Q, K, V의 관계를 정리해봅시다.

인코더의 첫번째 서브층 : Query = Key = Value

디코더의 첫번째 서브층 : Query = Key = Value

디코더의 두번째 서브층 : Query : 디코더 행렬 / Key = Value : 인코더 행렬

디코더의 두번째 서브층을 확대해보면, 다음과 같이 인코더로부터 두 개의 화살표가 그려져 있습니다.



두 개의 화살표는 각각 Key와 Value를 의미하며 이는 인코더의 마지막 층에서 온 행렬로부터 얻습니다. 반면 Query는 디코더의 첫번째 서브층의 결과 행렬로부터 얻는다는 점이 다릅니다. Query가 디코더 행렬, Key가 인코더 행렬일 때, 어텐션 스코어 행렬을 구하는 과정은 다음과 같습니다.

$$\begin{array}{c}
 \text{Q} \\
 \begin{array}{c}
 < sos > \\
 je \\
 suis \\
 \acute{e}tudiant
 \end{array}
 \end{array}
 \times K^T
 \begin{array}{c}
 I \text{ am a student} \\
 \begin{array}{c}
 I \text{ am a student}
 \end{array}
 \end{array}
 =
 \begin{array}{c}
 I \text{ am a student} \\
 \begin{array}{c}
 < sos > \\
 je \\
 suis \\
 \acute{e}tudiant
 \end{array}
 \end{array}
 \begin{array}{c}
 \text{Attention Score Matrix}
 \end{array}$$

전체 구성

Transformer의 전체 Architecture는 다음과 같습니다.

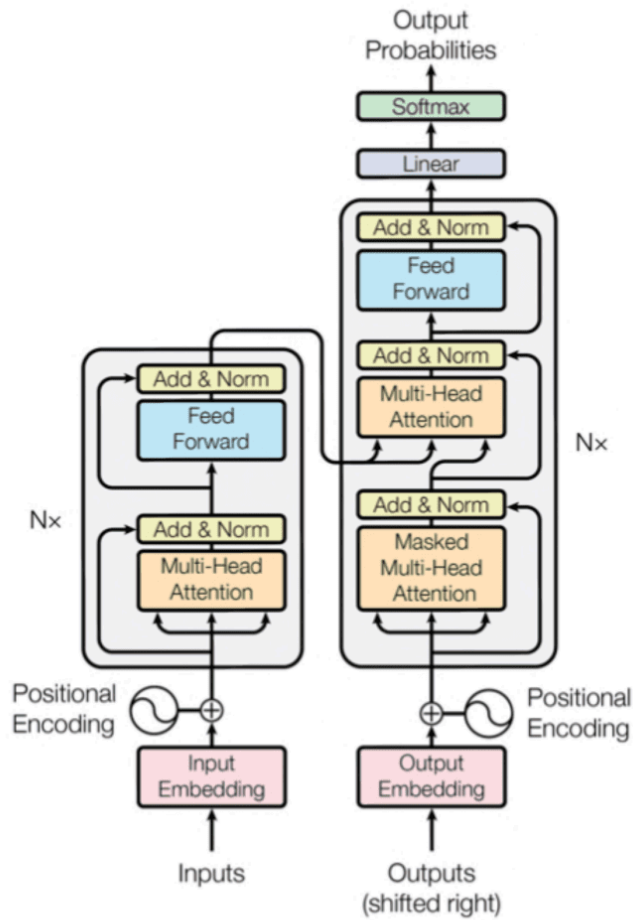


Figure 1: The Transformer - model architecture.

그림14. Transformer Architecture

먼저 왼쪽 하단의 Inputs는 Input Embedding을 거쳐 Positional Encoding을 더해주는 모습을 볼 수 있습니다. 그리고 나서 N번의 Encoding, 이어서 N번의 Decoding 과정을 거치고 Linear와 Softmax를 통해 최종 Output이 나오는 구조입니다.

5-2. Encoder

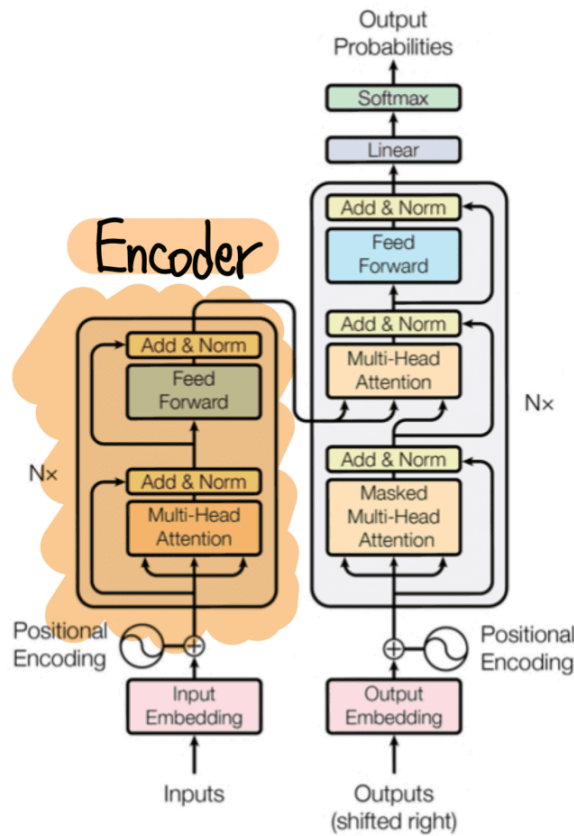


Figure 1: The Transformer - model architecture.

그림15. Transformer Encoder

위 그림과 같이 Transformer에서 입력 데이터를 Self Attention 연산을 통해 데이터를 변환하는 모듈을 Encoder 라고 합니다. 위에서 설명한 것과 같이 Multi Head Self Attention으로 구성되어 있으며 6개의 동일한 모듈이 직렬로 연결되어 있는 구조입니다. 이 때 그림에서 표현한 Multi Head Attention은 우리가 알고 있는 Self Attention 연산을 여러번 수행하는 구조를 말합니다. 아래에서 살펴볼 Decoder에서의 Multi Head Attention과 다른 부분이죠.

5-3. Decoder

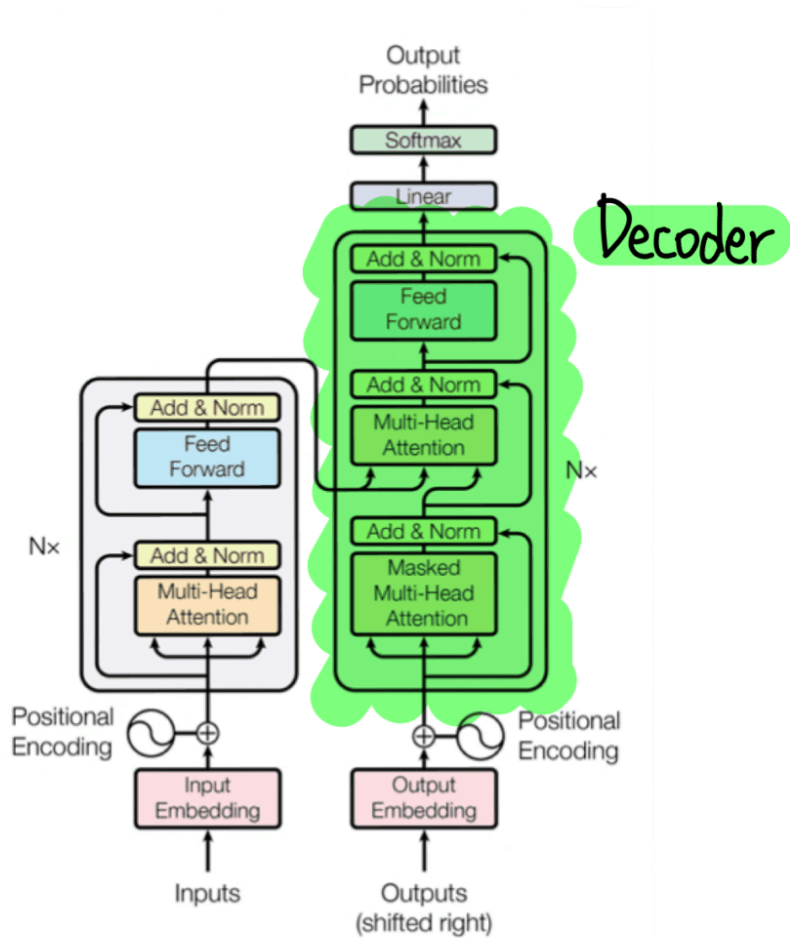


Figure 1: The Transformer - model architecture.

그림16. Transformer Decoder

위 그림과 같이 Encoding을 거친 입력 데이터를 출력 데이터와 같이 연산하여 최종 결과를 출력하는 모듈을 Decoder라고 합니다. 이때 Decoder에는 두 가지 Self Attention 변형 버전이 포함되어 있는데요.

첫 번째는 Masked Multi Head Attention입니다. Masked가 붙은 이유는 특정 데이터를 못 보도록 Masking 처리를 했기 때문인데요. Decoder는 Encoder와 달리 입력 데이터와 출력 데이터를 동시에 입력 받습니다. 번역 문제를 예로 들면 번역할 문장과 번역된 문장을 같이 입력 받습니다. 이때 번역할 문장은 정답에 해당하는데, 모두 입력 받으면 학습을 하는 의미가 없겠죠? 따라서 번역된 문장에서 미래 정보 부분은 Masking 처리해 줍니다. 번역은 순차적으로 출력되기에, 내가 번역해야 하는 단어부터는 모두 Masking 처리해주는 것이죠. 이렇게 Masking을 한 뒤 번역된 정보에 대해 Self Attention 연산을 수행합니다.

두 번째는 Cross Self Attention 입니다. 위 그림에서는 Decoder의 Multi Head Attention이라고 표현된 부분인데요. 이 모듈은 입력으로 입력 (번역할 정보)와 출력 (번역된 정보)를 모두 받는데요. 이 두 가지 정보의 관계를 고려하여 Self Attention을 수행해야 합니다. 따라서 Query와 Key는 번역할 정보로, Value는 번역된 정보로 구성합니다. 자기

자신만을 사용한 Attention이 아니므로 엄밀히 말하면 Self Attention은 아니라고 할 수 있습니다.

5-4. Dense & Softmax

마지막으로 Linear 연산과 Softmax 연산을 거쳐 최종 데이터를 출력해줍니다.